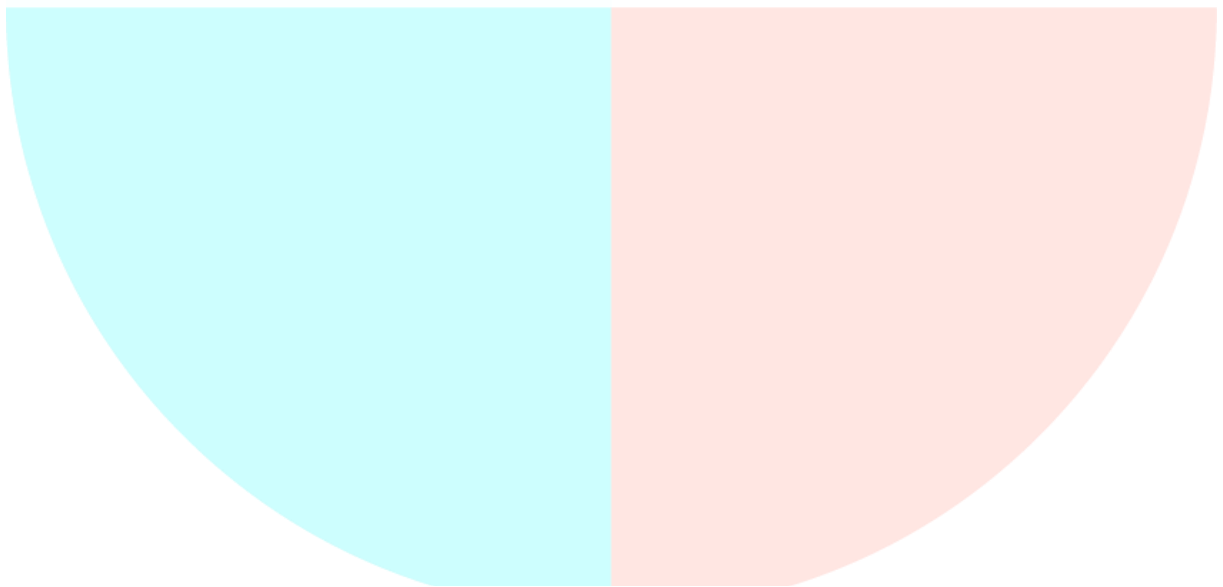


Bloque 2 - Tema 4

SISTEMAS OPERATIVOS. CARACTERÍSTICAS Y
ELEMENTOS CONSTITUTIVOS. SISTEMAS
WINDOWS. SISTEMAS UNIX Y LINUX. SISTEMAS
OPERATIVOS PARA DISPOSITIVOS MÓVILES

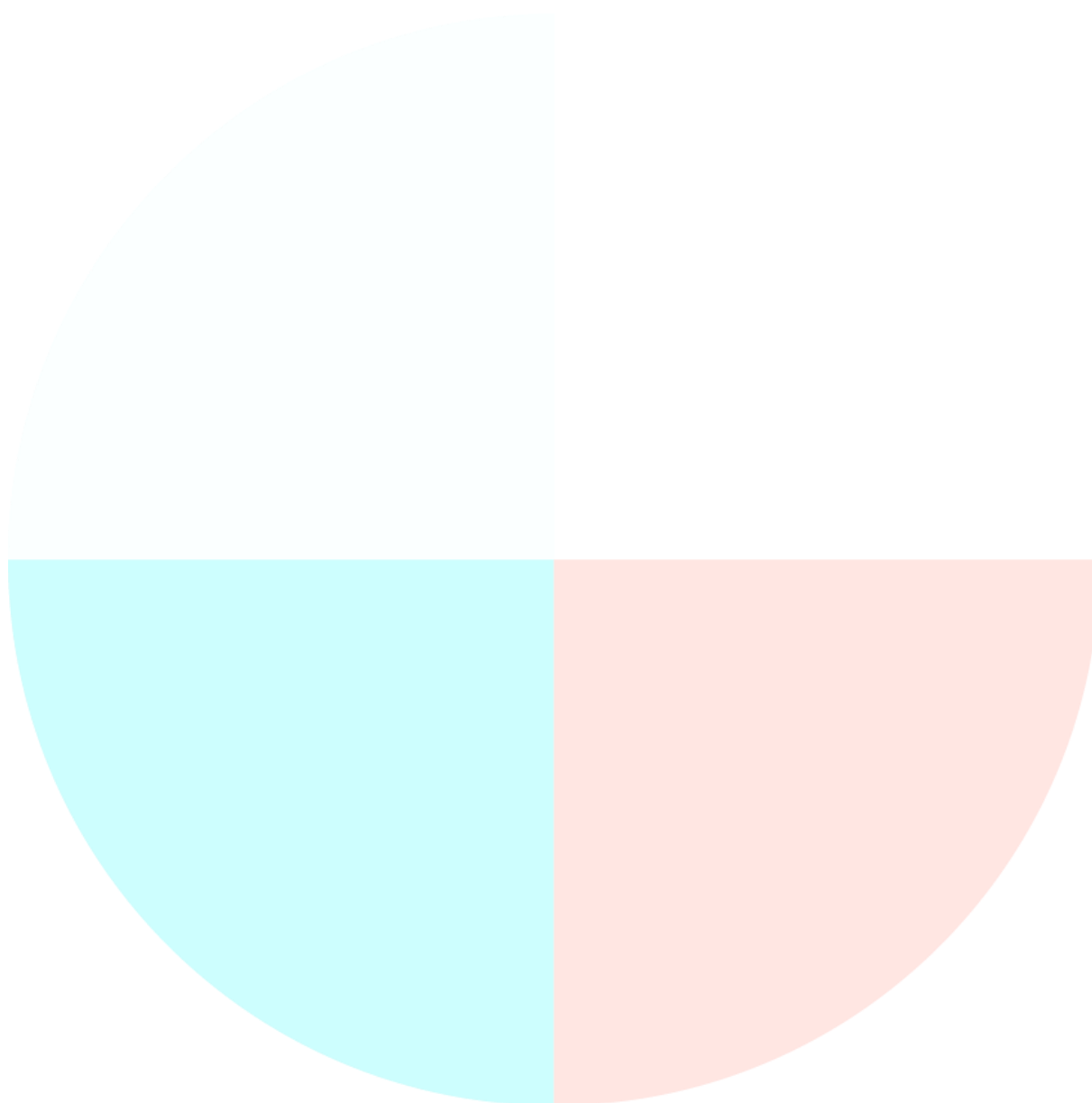


PREPARACIÓN OPOSICIONES
TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE	2
1. DEFINICIÓN SISTEMA OPERATIVO.....	4
2. ESTRUCTURA DE UN SISTEMA OPERATIVO	5
1. Estructura monolítica.....	5
2. Estructura jerárquica o en capas.....	6
3. Cliente-servidor (microkernel).....	6
4. Máquina virtual.....	7
3. COMPONENTES DEL SISTEMA OPERATIVO	8
4. CLASIFICACIÓN SS.OO.	9
1. SS.OO. por SERVICIOS.....	9
2. SS.OO. por la FORMA de ofrecer sus servicios.....	10
5. ADMINISTRACIÓN DE PROCESOS	11
1. Planificación del procesador	12
2. Algoritmos de planificación.....	14
3. Problemas de la concurrencia	16
6. ADMINISTRACIÓN DE LA MEMORIA.....	17
1. Gestión simple.....	17
2. Multiprogramación	18
3. Multiprogramación en memoria real.....	19
4. Multiprogramación en memoria virtual.....	20
7. ADMINISTRACIÓN DEL SISTEMA DE E/S	27
8. ADMINISTRACIÓN DE ARCHIVOS	29
9. SISTEMAS DE FICHEROS EXISTENTES.....	31
10. ARQUITECTURAS DE SISTEMAS OPERATIVOS	37
1. Interoperabilidad de procesos en Windows	37
11. SISTEMAS WINDOWS.....	38
1. Familia Windows: puestos de usuario.....	38
2. Active Directory.....	41
3. Línea de comandos.....	43
4. Modos de funcionamiento	43
5. Métodos de autenticación en Windows.....	44
6. Registro de Windows	44
7. Microsoft Azure.....	46
12. SISTEMAS UNIX Y LINUX	51
1. Distribuciones GNU/Linux	51
2. Entornos de escritorio y shells.....	53
3. Aspectos generales	54
4. Modos de funcionamiento	56
5. Procesos	57
6. Estructura de directorios.....	63
7. Comandos sistema de ficheros.....	65
8. Flujos de información.....	71
9. Permisos de archivo y directorio	72
13. SISTEMAS OPERATIVOS PARA DISPOSITIVOS MÓVILES.....	79
1. Android.....	79
2. iOS.....	86
3. Windows 10 Mobile	87
4. Firefox OS.....	87
5. KaiOS.....	87
6. Ubuntu Touch.....	88
7. MeeGo.....	88
8. Tizen.....	88

9.	<i>HarmonyOS</i>	88
10.	<i>Sailfish OS</i>	88
11.	<i>Xiaomi HyperOS</i>	88
12.	<i>Otros SO móviles</i>	89



1. DEFINICIÓN SISTEMA OPERATIVO

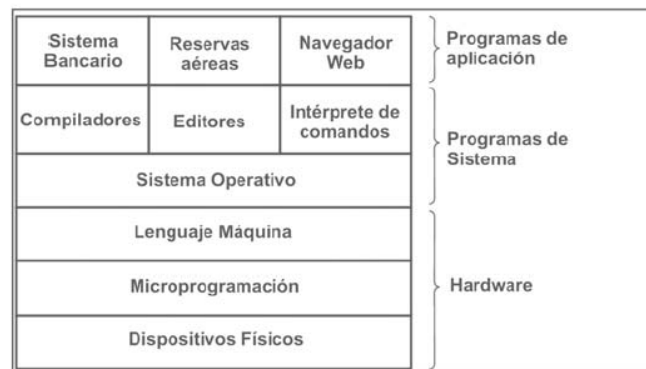
El software de un ordenador se puede dividir en dos clases:

- **Programas de sistema:** manejan la operación del ordenador.
- **Programas de aplicación:** resuelven problemas de los usuarios.

Dentro de los programas de sistema, el más importante es el **Sistema Operativo**, el cual controla todos los recursos del ordenador y ofrece la base sobre la cual pueden escribirse y ejecutarse programas de aplicación.

Como definición formal, podemos hacer uso de la formulada por Stallings:

Un **sistema operativo** es un programa que controla la ejecución de los programas de aplicación y que actúa como interfaz entre el usuario de un computador y el hardware del mismo.



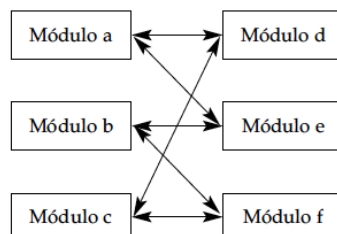
2. ESTRUCTURA DE UN SISTEMA OPERATIVO

Las estructuras disponibles para un sistema operativo son:

- Monolítica.
- Jerárquica o en capas.
- Cliente-servidor (microkernel).
- Máquina virtual.

1. Estructura monolítica

Los primeros sistemas operativos estaban formados por un **único programa** compuesto por un conjunto de rutinas entrelazadas de tal manera que cualquier rutina puede llamar a otra.

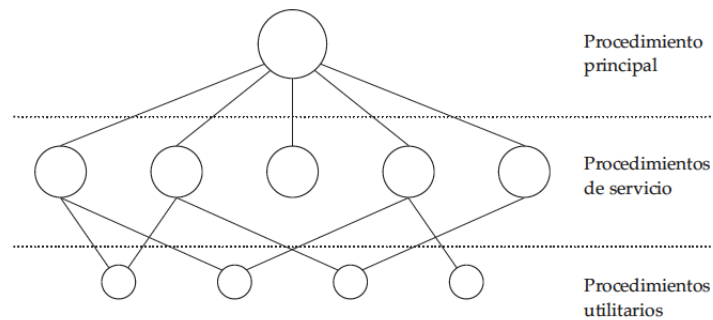


Se construye el programa final a partir de módulos compilados separadamente, por lo que es fundamental una buena definición de parámetros de enlace entre las distintas rutinas. Este hecho provoca un gran acoplamiento.

Los sistemas operativos monolíticos carecen de protección y privilegios al acceder a los recursos propios del ordenador (memoria, disco, CPU...).

Como gran ventaja, destaca su eficiencia y rapidez en su ejecución, ya que están hechos a medida. Y como desventaja destacable se encuentra su nula flexibilidad.

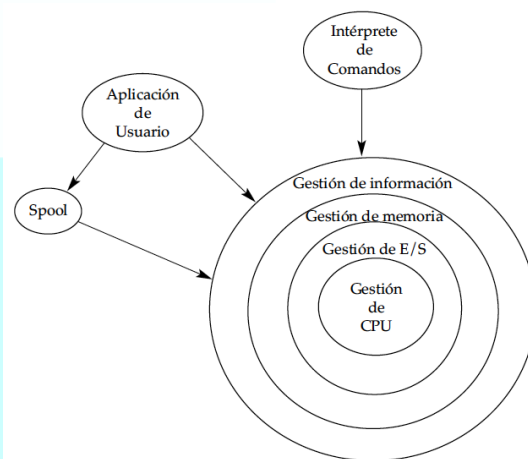
Para hacer llamadas al sistema operativo los servicios que presta éste se solicitan colocando los parámetros en un lugar definido (la pila), y ejecutando después una instrucción de llamada (TRAP). La instrucción cambia de modo usuario a modo kernel y transfiere el control al sistema operativo. Entonces, el sistema operativo obtiene los parámetros y determina qué llamada al sistema debe ejecutarse. Esta organización sugiere una estructura básica en donde un procedimiento principal invoca el procedimiento de servicio solicitado, el cual puede hacer uso de uno o varios procedimientos utilitarios que realizan tareas comunes a más de un procedimiento de servicio. Esta división de los procedimientos en tres capas se muestra en la figura siguiente.



2. Estructura jerárquica o en capas

Con el tiempo, al aumentar las necesidades de los usuarios se organiza internamente el sistema operativo, dividiéndose en pequeñas partes bien definidas y con una interfaz para ser usada por el resto de los elementos.

Así se obtiene una estructura jerárquica o de niveles. Ejemplos de este tipo de sistemas son THE, para fines didácticos y Multics y **Unix**.

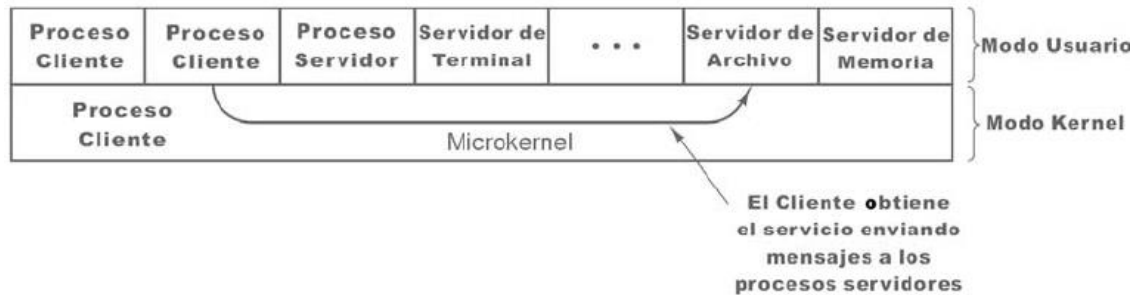


La estructura de la figura es la que se basan muchos sistemas operativos actuales. Cada capa solo tiene acceso a la inmediatamente superior e inferior. Así las capas internas están más protegidas de accesos indebidos desde capas externas, ya que las primeras se consideran más privilegiadas.

3. Cliente-servidor (microkernel)

El núcleo tiene como misión establecer la comunicación entre los clientes y los servidores. Los procesos pueden ser tanto servidores como clientes. Por ejemplo, un programa de aplicación normal es un cliente que llama al servidor correspondiente para acceder a un archivo o realizar una operación de entrada/salida sobre un dispositivo concreto. A su vez, un proceso cliente puede actuar como servidor para otro.

Este paradigma ofrece gran flexibilidad en cuanto a los servicios posibles en el sistema final, ya que el núcleo provee solamente funciones muy básicas de memoria, entrada/salida, archivos y procesos, dejando a los servidores proveer la mayor parte de los servicios que el usuario final o programador puede usar. Estos servidores deben tener mecanismos de seguridad y protección que, a su vez, serán filtrados por el núcleo que controla el hardware.

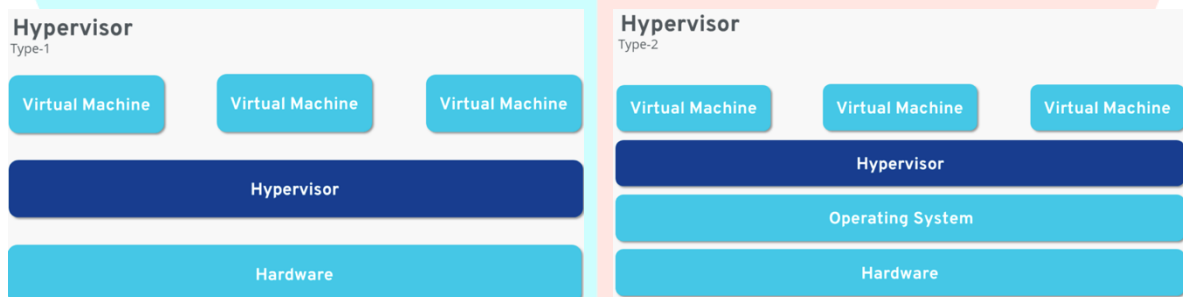


4. Máquina virtual

Se trata de un tipo de sistemas operativos que presentan una interfaz a cada proceso, mostrando una máquina que parece idéntica a la máquina real subyacente.

El objetivo de los sistemas operativos de máquina virtual es el de integrar distintos sistemas operativos dando la sensación de ser varias máquinas diferentes. El núcleo de estos sistemas operativos se denomina **monitor virtual** y tiene como misión llevar a cabo la multiprogramación, presentando a los niveles superiores tantas máquinas virtuales como se soliciten.

Estas máquinas virtuales son una réplica de la máquina real, de manera que en cada una de ellas se pueda ejecutar un sistema operativo diferente, que será el que ofrezca la máquina extendida al usuario.



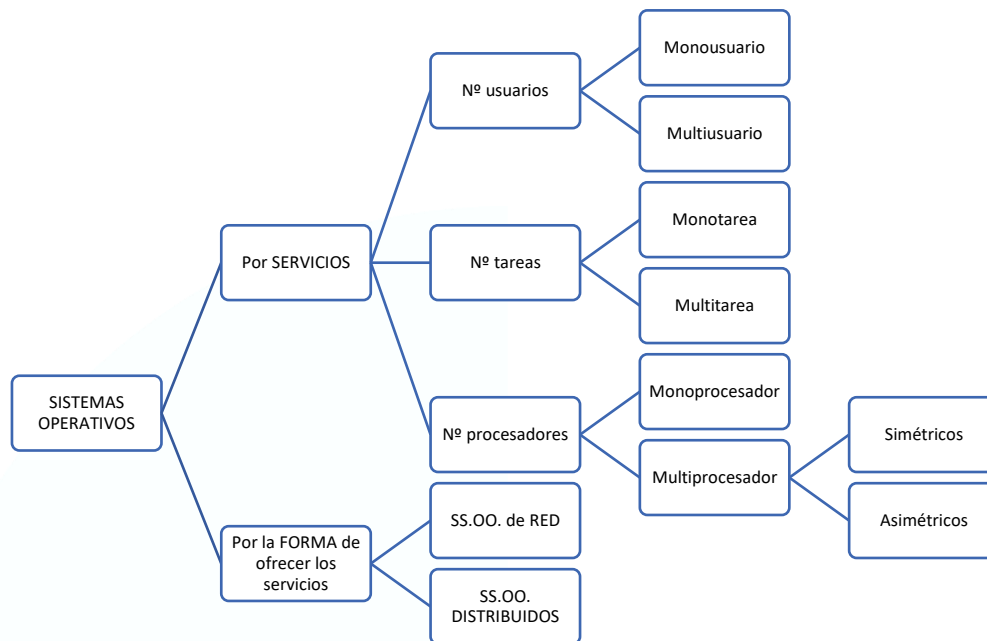
Ejemplos de este tipo de S.O. son Citrix XenServer, VMWare o VirtualBox.

3. COMPONENTES DEL SISTEMA OPERATIVO

Un sistema operativo está conformado básicamente por los siguientes componentes:

- **Administrador de procesos**: encargado de crear y eliminar procesos, suspender y reanudar la ejecución de procesos, proporcionar mecanismos de sincronización, de comunicación de procesos, de manejo de bloqueos.
- **Administrador de memoria principal**: controla las zonas de memoria utilizadas y quién las utiliza. Decide qué procesos se cargarán en memoria si queda espacio disponible. Asigna y recupera espacio no necesitado.
- **Administrador del sistema de E/S**: encargado de proporcionar manejadores para dispositivos de hardware específicos y de presentar una interfaz general común para interactuar con los diferentes manejadores.
- **Administrador de archivos**: es el encargado de la gestión del espacio de almacenamiento en disco. Se ocupa de los procesos relacionados con la creación y borrado de archivos, el establecimiento de la correspondencia entre archivos y almacenamiento secundario, la gestión de directorios y la gestión de copias de seguridad.
- **Sistema de protección**: encargado de controlar el acceso a los recursos, garantiza el acceso con la debida autorización y evita accesos no permitidos.
- **Sistema de comunicación**: encargado de gestionar los accesos a la red, facilita la conexión a través de la red de los distintos procesadores (en sistemas multiordenador). Permite el acceso a los recursos compartidos.

4. CLASIFICACIÓN SS.OO.



1. SS.OO. por SERVICIOS

Clasificación desde el punto de vista del usuario final:

- Por el número de usuarios:
 - **Monousuario:** soportan un único usuario a la vez, sin importar el número de procesadores y de procesos/tareas a ejecutar simultáneamente.
 - **Multiusuario:** admiten varios usuarios simultáneos.
- Por el número de tareas:
 - **Monotarea:** solo permite una tarea por usuario. Se puede dar el caso de sistemas multiusuario y monotarea.
 - **Multitarea:** permite al usuario ejecutar realizar varias tareas al mismo tiempo.
- Por el número de procesadores:
 - **Monoprocesador:** el sistema operativo solo es capaz de manejar un procesador.
 - **Multiprocesador:** ordenador con más de un procesador en el que el sistema operativo es capaz de utilizar todos para distribuir la carga.
 - **Simétrico:** los procesadores comparten la misma memoria. Según el modo de gestionar la memoria compartida, tenemos: UMA (acceso uniforme a memoria) o NUMA (acceso no uniforme a memoria).

- **Asimétrico:** un procesador "maestro" donde se ejecuta el S.O. y otros "esclavos" que ejecutan tareas. La gestión de memoria es distribuida (MMD), es decir, cada procesador dispone de una memoria propia.

2. SS.OO. por la FORMA de ofrecer sus servicios

Esta clasificación se refiere a una visión externa, a cómo el usuario accede a los servicios.

Sistemas Operativos en RED

Los sistemas operativos de red se definen como aquellos que tienen la capacidad de interactuar con sistemas operativos en otras computadoras a través de un medio de transmisión con el objeto de intercambiar información, transferir archivos, ejecutar comandos remotos y un sin fin de otras actividades.

El punto crucial de estos sistemas es que el usuario debe saber la sintaxis de un conjunto de comandos o llamadas al sistema para ejecutar estas operaciones, además de la ubicación de los recursos a los que quiere acceder.

Sistemas Operativos DISTRIBUIDOS

Los sistemas operativos distribuidos abarcan los mismos servicios que los sistemas operativos de red, logrando integrar recursos (impresoras, unidades de respaldo, memoria, procesos, unidades centrales de proceso) en una sola máquina virtual a la que el usuario accede de forma transparente. Es decir, ahora el usuario ya no necesita saber la ubicación de los recursos, sino que los conoce por nombre y simplemente los usa como si todos ellos fuesen locales a su lugar de trabajo habitual.

Todo lo anterior es el marco teórico de lo que se desearía tener como sistema operativo distribuido, pero en la realidad no se ha conseguido crear uno completamente funcional, por la complejidad que suponen: distribuir los procesos en las varias unidades de procesamiento, reintegrar sub-resultados, resolver problemas de concurrencia y paralelismo, recuperarse de fallas de algunos recursos distribuidos y consolidar la protección y seguridad entre los diferentes componentes del sistema y los usuarios.

5. ADMINISTRACIÓN DE PROCESOS

Un PROCESO es un programa en ejecución. Incluye los valores que tienen el contador de programa, los registros y las variables.

La diferencia entre proceso y programa es fundamental. Mientras un programa es un conjunto de instrucciones cuyo objetivo es la realización de una tarea, un proceso es una actividad. Así, varios procesos pueden compartir el mismo programa.

CREACIÓN de procesos

Hay cuatro sucesos principales que causan la creación de procesos:

- **Inicialización del sistema:** el arranque de un sistema operativo conlleva la creación de varios procesos (interactivos o en segundo plano).
- **Ejecución de una llamada al sistema** por parte de un proceso en ejecución.
- **Solicitud de un usuario para crear un proceso:** en los sistemas interactivos, los usuarios pueden iniciar un programa.
- **Inicio de un trabajo por lotes:** aplicable a los mainframes con sistemas por lotes. Los usuarios envían una lista de trabajos y el S.O. los ejecuta adecuadamente.

TERMINACIÓN de procesos

Pueden terminar debido a las siguientes condiciones:

- **Terminación normal** (voluntaria): el proceso terminó la tarea que debía realizar.
- **Terminación por error** (voluntaria): error controlado que produce la finalización.
- **Error fatal** (involuntaria): error no esperado e incontrolado y el S.O. cancela su ejecución.
- **Terminado por otro proceso** (involuntaria): un proceso solicita la finalización de otro.

JERARQUÍA de procesos

Algunos sistemas operativos permiten establecer unas relaciones jerárquicas entre procesos. Así, cuando un proceso “padre” crea otro proceso, denominado proceso “hijo”, se crea una jerarquía, permitiéndose actuar de manera conjunta con dicha jerarquía.

ESTADOS de un proceso

Los estados de un proceso obedecen a su participación y disponibilidad dentro del S.O. y surgen de la necesidad de controlar la ejecución de cada proceso. Los procesadores sólo pueden ejecutar un sólo proceso a la vez, turnándolos para el uso de éste.

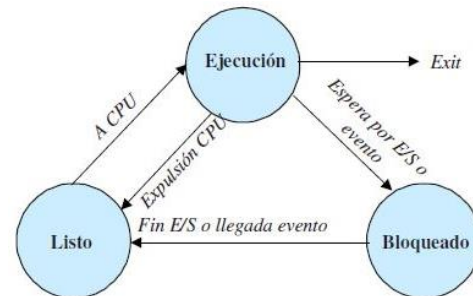
Los posibles estados que puede tener un proceso son:

- **En ejecución:** es un proceso que está haciendo uso del procesador.

- **Listo**: proceso que puede ejecutarse, pero está detenido para que el procesador pueda ejecutar otro proceso.
- **Bloqueado**: no puede ejecutarse mientras no ocurra cierto suceso externo a él.

Son posibles 5 transiciones entre estos estados:

- Transición 1: Listo – Ejecución.
- Transición 2: Ejecución – Bloqueado
- Transición 3: Ejecución – Listo.
- Transición 4: Bloqueado – Listo.
- Transición 5: Ejecución – Final.



MODELO de procesos

La gestión del conjunto de procesos se implementa a través de la **TABLA de PROCESOS**. Esta tabla contiene una entrada para cada proceso en la que se incluye su estado y la información necesaria para continuar la ejecución del mismo en el punto donde se interrumpió. Esta estructura de datos también se conoce como **Bloque de Control del Proceso (PCB, Process Control Block)**.

1. Planificación del procesador

La planificación del procesador se refiere a la manera o técnica que se usa para decidir cuánto tiempo de ejecución y cuándo se le asigna a cada proceso del sistema. Obviamente, si el sistema es monousuario y monotarea no hay mucho que decidir, pero en el resto de los sistemas esto es crucial para el buen funcionamiento del sistema.

En los sistemas de planificación generalmente se identifican tres niveles de planificación:

- El **planificador a LARGO PLAZO: long term scheduler**. Decide que trabajos (conjunto de procesos) son candidatos para convertirse en procesos listos para competir por los recursos del sistema.
- El **planificador a MEDIO PLAZO: mid term scheduler**. Decide qué procesos se suspenden o se reanudan para lograr ciertas metas de rendimiento.
- El **planificador a CORTO PLAZO: dispatcher o short term scheduler**. Decide qué proceso, de los que ya están listos/preparados, es al que le corresponde estar ejecutándose en la unidad central de proceso.

Una estrategia de planificación debe buscar que los procesos obtengan sus turnos de ejecución apropiadamente, conjuntamente con un buen rendimiento y minimización de la sobrecarga (overhead) del planificador mismo. En general, se buscan **5 objetivos** principales:

- **Justicia o Imparcialidad**: todos los procesos son tratados de la misma forma, y en algún momento obtienen su turno de ejecución o intervalos de tiempo de ejecución hasta su terminación exitosa.
- **Maximizar la producción** (throughput): el sistema debe de finalizar el mayor número de procesos por unidad de tiempo.
- **Minimizar el tiempo de respuesta** (turnaround): cada usuario o proceso debe observar que el sistema les responde consistentemente a sus requerimientos.
- **Evitar el aplazamiento indefinido** (inanición): los procesos deben terminar en un plazo finito de tiempo.
- El **sistema debe ser predecible**: ante cargas de trabajo ligeras el sistema debe responder rápido y con cargas pesadas debe ir degradándose paulatinamente, es decir, bajo cargas similares la respuesta ha de ser similar.

Definiciones sobre rendimiento:

- **Turnaround**: tiempo transcurrido entre que se lanza un proceso hasta que se completa.
- **Throughput**: número de procesos finalizados por unidad de tiempo o también cantidad de trabajo útil ejecutado por unidad de tiempo.
- **Capacidad**: máxima cantidad de trabajo útil ejecutado por unidad de tiempo.
- **Tamaño del quantum**: cantidad de tiempo ininterrumpido de la CPU que un sistema de tiempo compartido asigna a los diferentes procesos.

Las **características** de los procesos que se deben tener en cuenta al ser planificados son:

- **Cantidad de E/S**: proceso que realiza un número elevado de operaciones E/S.
- **Cantidad de uso de CPU**: proceso que usa intensivamente la CPU.
- **Proceso de lote**: recibe los datos de ficheros.
- **Proceso interactivo**: recibe los datos de usuarios del sistema.
- **Proceso en tiempo real**: la respuesta se realizará en un tiempo máximo determinado.
- **Longevidad del proceso**: tiempo necesario para realizar la tarea encomendada.

Los 2 **tipos de planificación** que existen son:

- **Planificación NO APROPIATIVA (no expulsiva)**: es aquella que cuando un proceso está en ejecución ya no puede ser suspendido, no se le puede expulsar de la CPU. Como problema principal tenemos la existencia de **ciclos infinitos** que aplazaría indefinidamente al resto de procesos (inanición) o también que un proceso de alta prioridad tendrá que esperar a la finalización del proceso actual.
- **Planificación APROPIATIVA (expulsiva)**: es aquella en la que el planificador puede expulsar a un proceso de la CPU para ejecutar otro proceso. De esta forma, cada vez que el reloj lanza una interrupción periódica el planificador decide si el proceso en ejecución continúa o se sustituye por otro. Al cambio de un proceso por otro se le conoce como **cambio de contexto**.

2. Algoritmos de planificación

Los algoritmos de planificación son:

- **SJF (Shortest Job First)**: el trabajo más corto primero. **NO APROPIATIVO**
- **SRTF (Shortest Remaining Time First)**: tiempo restante más corto. **APROPIATIVO**
- **FCFS (First Come First Served)**: primero en llegar, primero en ejecutarse. **NO APROPIATIVO**
- **Round Robin (Turno circular)**. **APROPIATIVO**
- **Por prioridad**. **APROPIATIVO** o **NO APROPIATIVO**
- **Colas multinivel**. **APROPIATIVO**

SJF

Se requiere disponer de una estimación del tiempo que un proceso ha de estar en ejecución. Tras esto, se ejecutan primero aquellos procesos que necesitan menos tiempo. Su planificación es no apropiativa. Este algoritmo obtiene el mejor tiempo de respuesta promedio.

Ejemplo: si llegan 5 procesos A, B, C, D y E cuyos tiempos de CPU son 26, 18, 24, 12 y 4 unidades de tiempo, se observa que el orden de ejecución será E, D, B, C y A (4, 12, 18, 24 y 26 unidades de tiempo respectivamente).

PROCESO	DURACIÓN	LLEGADA	COMIENZA	TERMINA
A	26	0	62	88
B	18	0	18	36
C	24	0	37	61
D	12	0	5	17
E	4	0	0	4

SRTF

Es similar al algoritmo SJF, pero su planificación es apropiativa. Se ejecuta primero el proceso que le queda menos por ejecutar. El proceso en CPU es desalojado si llega a la cola un proceso con duración más corta.

Ejemplo: Caso anterior modificado aplicando este algoritmo.

PROCESO	LLEGADA	DURACIÓN
A	0	10
B	8	9
C	1	2
D	0	5
E	4	3

[illegible]

FCFS

El primero que llega es el primero que se ejecuta, es decir, los procesos reciben su turno conforme llegan. Dicho de otra forma, el dispatcher elige al proceso que lleva más tiempo listo. Su funcionamiento es similar a una cola tipo FIFO.

Como ventaja citar que este algoritmo es justo y no provoca aplazamiento indefinido. Como desventaja es que no aprovecha ninguna característica de los procesos.

Ejemplo: dado el conjunto de procesos siguiente, utilizamos este algoritmo de planificación.

PROCESO	LLEGADA	DURACIÓN
A	0	5
B	4	2
C	1	7
D	2	1
E	3	8

A	A	A	A	A	C	C	C	C	C	C	D	E	E	E	E	E	E	E	B	B
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Round Robin

Consiste en darle a cada proceso un intervalo de tiempo de ejecución (denominado **quantum**) y al finalizar dicho intervalo se da el turno a otro proceso.

Los procesos están ordenados en una cola circular de tal forma que cuando un proceso termina su quantum vuelve al final de la cola.

Ejemplo: aplicación de este algoritmo, siendo el quantum $q=2$.

PROCESO	LLEGADA	DURACIÓN
A	0	5
B	4	2
C	1	7
D	2	1
E	3	8

A	A	C	C	D	A	A	E	E	B	B	C	C	A	E	E	C	C	E	E	C	E	E
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Por prioridad

Los procesos de mayor prioridad se ejecutan primero. Para procesos con la misma prioridad se aplica FCFS o Round Robin. La planificación puede ser apropiativa o no apropiativa.

La ventaja de este algoritmo es que es flexible. Como desventaja está el aplazamiento indefinido para procesos de baja prioridad.

Colas multinivel

Los procesos se clasifican en diferentes grupos (por ejemplo, procesos interactivos y procesos por lotes) usando una cola para cada una de las agrupaciones. A cada grupo de procesos concreto se le asigna una prioridad distinta y un algoritmo de planificación diferente.

3. Problemas de la concurrencia

En los sistemas concurrentes o de tiempo compartido los procesos compiten por los recursos del sistema, pudiendo generarse si no se tratan debidamente una serie de problemas:

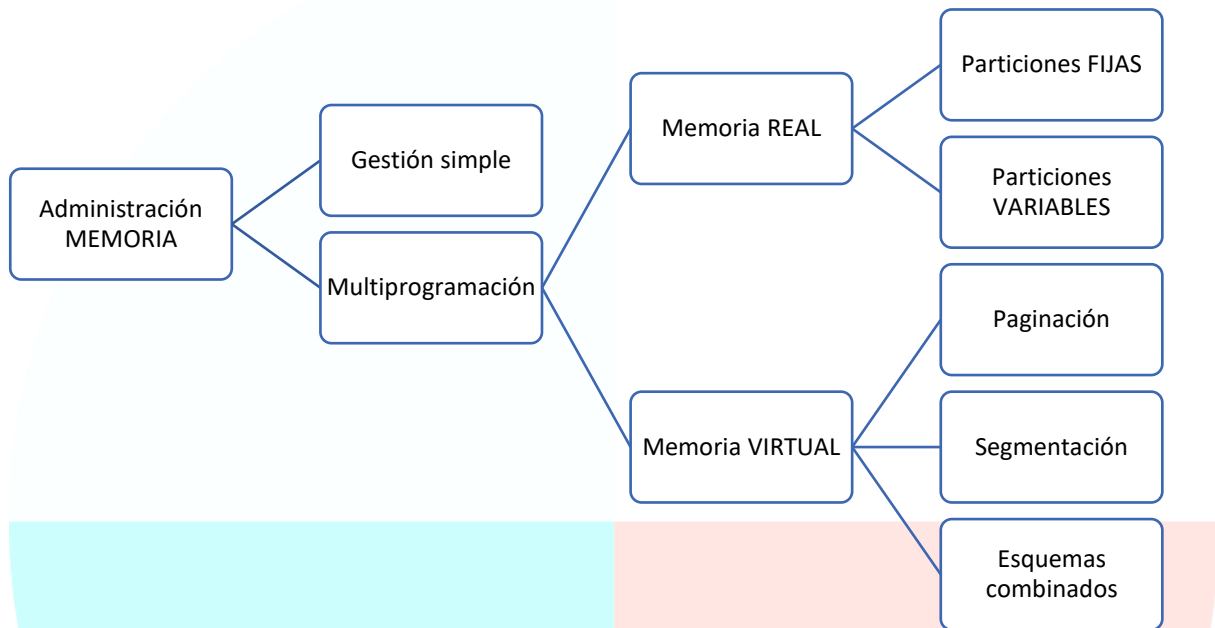
- **Aplazamiento indefinido (inanición):** sucede cuando un proceso es postergado continuamente sin posibilidad de finalizar su ejecución.
- **Condición de carrera:** ocurre cuando dos o más procesos acceden a un recurso sin control de tal forma que el resultado depende del orden de ejecución.
- **Condición de espera circular:** sucede cuando en un conjunto de procesos, cada uno de ellos tiene asignado algún recurso y necesita acceder a otros que están bloqueados por otros procesos de ese conjunto.
- **Condición de espera ocupada:** sucede cuando un proceso solicita un recurso previamente asignado a otro proceso y tiene que esperar a que voluntariamente libere dicho recurso (esto último se conoce como condición de no apropiación).
- **Condición de ocupar y esperar un recurso:** ocurre cuando un proceso tiene asignado un recurso y sin liberarlo solicita la asignación de otro.

6. ADMINISTRACIÓN DE LA MEMORIA

Los objetivos del sistema de gestión de memoria son:

- Ofrecer a cada proceso un espacio lógico propio.
- Proporcionar protección entre los procesos.
- Permitir que los procesos compartan memoria.

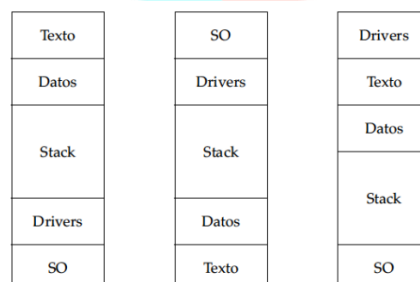
Vamos a ver las técnicas más usuales para el manejo de la memoria:



1. Gestión simple

Este esquema se usa principalmente en sistemas monousuario y monotarea, como MSDOS.

Bajo este esquema, la memoria real se utiliza para almacenar el programa que se esté ejecutando en un momento dado, con la visible desventaja de estar limitado a la cantidad de RAM disponible. La organización física bajo este esquema es muy simple: el sistema operativo se ubica en las localidades superiores o inferiores de la memoria, seguido por algunos manejadores de dispositivos (drivers).



Esto deja un espacio contiguo de memoria disponible que es utilizado por los programas del usuario, dejando generalmente la ubicación de la pila (stack) al último, con el objetivo de que ésta pueda crecer hasta el máximo posible. Estas diferentes opciones se pueden ver en la figura. Como es obvio, bajo estos esquemas no se requieren algoritmos sofisticados para asignar la memoria a los diferentes procesos, ya que éstos son ejecutados secuencialmente conforme van terminando los anteriores.

2. Multiprogramación

El concepto de multiprogramación consiste en la idea de poner en la memoria física más de un proceso al mismo tiempo de manera que si el que se está ejecutando se bloquea o pasa a estado Listo, se puede tomar otro proceso para que haga uso de la CPU.

De esta forma, la memoria física se divide en secciones de tamaño suficiente para contener a varios programas.

Antes de continuar nos detenemos en dos problemas: la **relocalización** y la **protección**.

Relocalización

Este problema consiste en que los programas que necesitan cargarse en la memoria real ya están compilados y enlazados, de manera que, internamente, contienen una serie de referencias a direcciones de instrucciones, rutinas y procedimientos que ya no son válidas en el espacio de direcciones de memoria real de la sección en la que se carga el programa.

Esto es, cuando se compiló el programa se definieron o resolvieron las direcciones de memoria de acuerdo a la sección de ese momento, pero si el programa se carga en una sección diferente, las direcciones reales ya no coinciden. En este caso, el manejador de memoria puede solucionar el problema de dos maneras: de manera «estática» o de manera «dinámica».

La solución «estática» consiste en que todas las direcciones del programa se vuelvan a recalcular al momento en que el programa se carga a memoria, esto es, prácticamente se vuelve a recompilar el programa.

La solución «dinámica» consiste en tener un registro que guarde la dirección base de la sección que va a contener al programa. Cada vez que el programa haga una referencia a una dirección de memoria, se le suma el registro base para encontrar la dirección real.

Ejemplo: supongamos que el programa se carga en una sección que comienza en la dirección 100. El programa hará referencias a las direcciones 50, 52, 54. Pero el contenido de esas direcciones no es el deseado, sino las direcciones 150, 152 y 154, ya que ahí comienza el programa. La suma de $100 + 50$, etcétera se hace en tiempo de ejecución.

La primera solución vale más la pena que la segunda si el programa contiene ciclos y es largo, ya que consumirá menos tiempo en la resolución inicial que la segunda solución en las resoluciones durante la ejecución.

Protección

Este problema se refiere a que, una vez que un programa ha sido cargado en memoria en alguna sección en particular, nada le impide al programador direccionar (por error o deliberadamente) posiciones de memoria menores que el límite inferior de su programa o superiores a la dirección mayor, es decir, que quiera referenciar localidades fuera de su espacio de direcciones. Obviamente, éste es un problema de protección, ya que no es legal leer o escribir en áreas de otros programas.

La solución a este problema también puede ser el uso de un **registro BASE** y un **registro LÍMITE**. El registro base contiene la dirección del comienzo de la sección que contiene al programa, mientras que el registro límite contiene la dirección donde termina. Cada vez que el programa hace una referencia a memoria, se comprueba si ésta pertenece al espacio de direcciones entre su registro base y su registro límite, si no es así se envía un mensaje de error y se aborta el programa.

Registro base <= proceso <= registro límite

3. Multiprogramación en memoria real

En el esquema de la multiprogramación en memoria real se manejan dos alternativas para asignarle a cada programa su partición correspondiente: particiones de tamaño fijo o particiones de tamaño variable.

Particiones FIJAS

La alternativa más simple son las particiones fijas. Dichas particiones se crean cuando se enciende el equipo y permanecen con los tamaños iniciales hasta que el equipo se reinicie.

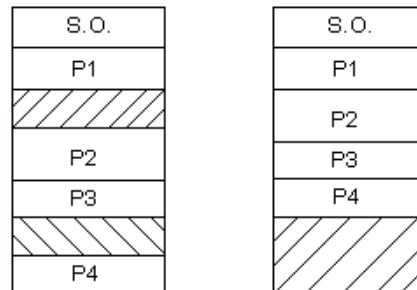
Es una alternativa muy vieja, las particiones se realizaban por el operador analizando los tamaños estimados de los trabajos de todo el día.

Problemas: trabajos de un tamaño superior al tamaño de la partición.

Particiones VARIABLES

La otra alternativa, que surgió después y como necesidad de mejorar la alternativa anterior, fue crear particiones contiguas de tamaño variable. Para esto, el sistema debía mantener ya una estructura de datos suficiente para saber en dónde había huecos disponibles y de dónde a dónde había particiones ocupadas por programas en ejecución.

Así, cuando un programa requería ser cargado en memoria, el sistema analizaba los huecos para saber si había alguno de tamaño suficiente para el programa entrante, si era así, le asignaba el espacio. Si no, intentaba relocalizar los programas existentes con el propósito de hacer contiguo todo el espacio libre y así obtener un hueco de tamaño suficiente. Si aun así el espacio era insuficiente, el programa se bloqueaba y el sistema optaba por otro. Al proceso mediante el que se juntan los huecos se le denomina de **compactación**.



Recubrimientos (overlays): permite albergar procesos con tamaño mayor que la memoria disponible. Consiste en que cuando no todo el proceso tiene que estar en MP ejecutándose, solo están en MP aquellos datos o instrucciones que puedan ser utilizados en un momento dado, para que el proceso se pueda ejecutar.

Para la asignación de programa a un hueco existen varios algoritmos:

- **Primer ajuste:** se asigna el primer hueco que sea mayor al tamaño deseado.
- **Mejor ajuste:** se asigna el hueco cuyo tamaño exceda en la menor cantidad al tamaño deseado. Requiere de una búsqueda exhaustiva.
- **Peor ajuste:** se asigna el hueco cuyo tamaño exceda en la mayor cantidad al tamaño deseado. Requiere también de una búsqueda exhaustiva.
- **El siguiente ajuste:** es igual que el primer ajuste con la diferencia que se deja un apuntador al lugar en donde se asignó el último hueco para realizar la siguiente búsqueda a partir de él.
- **Ajuste rápido:** se mantienen listas enlazadas separadas de acuerdo a los tamaños de los huecos, para así buscarle a los procesos un hueco más rápido en la cola correspondiente.

4. Multiprogramación en memoria virtual

La necesidad ejecutar programas grandes y el crecimiento en potencia de las CPUs empujaron a los diseñadores de los sistemas operativos a implantar un mecanismo para ejecutar automáticamente programas más grandes que la memoria real disponible, esto es, ofrecer memoria virtual.

La **memoria virtual** se llama así porque el programador ve una cantidad de memoria mucho mayor que la real. En realidad, se trata de la suma de la memoria de almacenamiento primario (memoria real) y una cantidad determinada de almacenamiento secundario (discos).

El sistema operativo, en su módulo de manejo de memoria, se encarga de intercambiar programas enteros, segmentos o páginas entre la memoria real y el medio de almacenamiento secundario. Si lo que se intercambia son procesos enteros, se habla entonces de multiprogramación en memoria real, pero si lo que se intercambian son segmentos o páginas, se puede hablar de multiprogramación con memoria virtual.

Esquemas:

- Paginación.
- Segmentación.
- Esquema combinado.

PAGINACIÓN pura

La paginación pura en el manejo de memoria consiste en que el sistema operativo divide dinámicamente los programas en unidades de tamaño fijo, los cuales va a intercambiar entre la memoria principal y el disco.

Al proceso de intercambiar páginas, segmentos o programas completos entre RAM y disco se le conoce como intercambio o **swapping**.

En la paginación, se debe elegir cuidadosamente el tamaño de las páginas, ya que si éstas son muy pequeñas el control por parte del sistema operativo para saber cuáles están en memoria principal y cuáles en disco, sus direcciones reales, etcétera, crece y provoca mucha sobrecarga. Por otro lado, si las páginas son muy grandes, la sobrecarga disminuye, pero entonces puede ocurrir que se desperdicie memoria en procesos pequeños, conociéndose como **fragmentación interna**. Debe haber un equilibrio.

Una ventaja de la paginación es que ya no se requiere que los programas estén ubicados en zonas de memoria adyacente, ya que las páginas pueden estar ubicadas en cualquier lugar de la memoria RAM.

La **MMU**¹ (Memory Management Unit) divide el programa en bloques de igual tamaño (en general una potencia de dos):

- La memoria física (MP) se divide en bloques denominados **marcos de página**.
- El espacio lógico se divide en bloques denominados **páginas**.

Para ejecutar un programa con n páginas, necesitamos m marcos libres (el sistema operativo se encarga de gestionar los marcos libres mediante un mapa de bits o una lista de marcos libres). m puede ser menor que n, esto es, podemos ejecutar un programa m mayor que la memoria principal disponible.

¹ MMU: dispositivo hardware que traduce direcciones virtuales en direcciones físicas (más eficiente que la traducción vía software).

Por otro lado, para cada proceso el sistema operativo mantiene una tabla auxiliar para la gestión de las páginas: la **TABLA DE PÁGINAS** (también llamada Tabla de Direcciones Virtuales). Cada entrada (PTE, Page Table Entry) en la **tabla de páginas del proceso** tiene una serie de campos:

- **Número de marco**: dirección real donde se encuentra la página.
- **Bit de presencia**: activado si la página está en MP.
- **Bit de modificado**: activado si la página ha sido modificada.
- **Bit de accedido**: activado si se ha accedido.
- **Propietario** de la página.
- **Permisos** sobre la página.

Uno de los aspectos más importantes de la paginación, así como de cualquier esquema de memoria virtual, es la forma de **traducir una dirección virtual a dirección real**. La traducción de direcciones virtuales a direcciones físicas consiste básicamente en conocer **dónde está el marco** en el que se encuentra cargada una página dada. De la traducción se encarga la MMU.



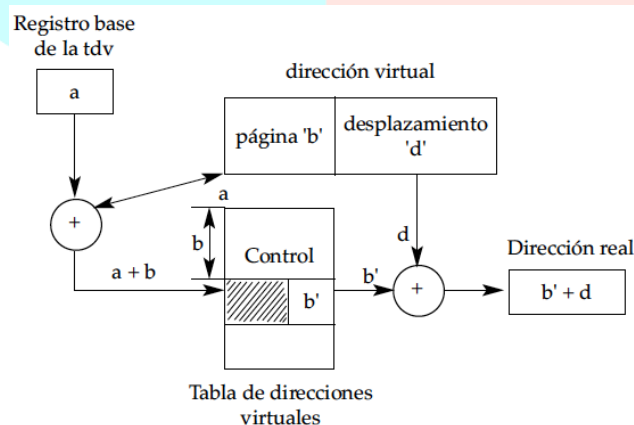
La traducción se realiza a través del PTE de la tabla de páginas del proceso, ya que contiene la dirección del marco donde está cargada.

La MMU descompone las direcciones lógicas/virtuales que le pasa la CPU en dos campos:

- **Número de página (b)**: se usa como índice para acceder a la b-ésima TP o TDV, que nos dará la dirección de la base del marco donde se encuentra la página en memoria.
- **Desplazamiento de página (d)**: sumado con la base de página define la dirección física que debe utilizar la MMU para acceder a la instrucción o dato.

Y posteriormente realiza la traducción de dirección virtual a dirección física.

Traducción: una dirección virtual $v' = (b, d)$ está formada por un número de página virtual b y un desplazamiento d .



Supongamos que el sistema ofrece un espacio de direcciones virtuales de 64 KB, con las siguientes consideraciones:

- Tamaño de página = 4 KB.
- Tamaño de la memoria principal = 32 KB.

Entonces tenemos 16 páginas virtuales y 8 páginas reales.

El dato que nos interesa ahora es el número de página real que le corresponde a la página virtual. Obviamente, de las 16 virtuales, sólo 8 tendrán un valor de control que dice que la página está cargada en MP, así como la dirección real de la página, denotada en la figura como b' .

Así, supóngase que para la página virtual número 14 la tabla dice que, efectivamente está cargada en MP y es la página real 2 (dirección de memoria 8.192). Una vez encontrada la página real, se le suma el desplazamiento, que es la dirección que deseamos dentro de la página buscada ($b' + d$).

Tras ver en el ejemplo con el procedimiento a seguir, podemos pensar que es muy costoso ya que requiere de varios accesos a disco (mucho más lento que a MP).

Para reducir el tiempo de acceso se introduce una caché hardware rápida denominada Búfer de reconocimiento de traducciones (**TLB**, Translation Look-aside Buffer). La traducción ahora se realiza de esta forma: si existe ya en el TLB, tenemos el marco. En caso contrario, la buscamos en la TP y ajustamos el TLB.

Además, hay que sumar el tiempo invertido en trasladar las páginas que pasan de memoria a disco. Para ello, se debe disponer de un **método** de elección para seleccionar la página candidata a salir de memoria. El objetivo de estos métodos es **minimizar la tasa de fallos de página**.

Tenemos varios algoritmos de REEMPLAZO de páginas:

- **Primera en entrar, primera en salir (FIFO)**: se descarga la página que haya entrado primera en memoria principal. Necesario disponer de un valor de control que almacene el momento en el tiempo que se cargó esa página en memoria. Es un algoritmo ineficiente, justo e imparcial.
- **La no usada recientemente (NRU)**: se elige la página que no se ha usado en el ciclo anterior. Se intenta aprovechar del principio de localidad del conjunto de trabajo (acceso a posiciones contiguas a la dirección ejecutada por la CPU).
- **La usada menos recientemente (LRU)**: parecido al anterior. Se elige la página que se usó hace más tiempo. Requiere búsqueda exhaustiva.
- **La no usada frecuentemente (NFU)**: en lugar de basarse en el tiempo se basa en el número de referencias. Elección de la página que no ha sido referenciada.
- **La menos frecuentemente usada (LFU)**: parecido al anterior. Búsqueda exhaustiva de la página menos referenciada.

Basada en
TIEMPO

Basada en
REFERENCIAS

- **Aleatoriamente:** se elige cualquier página sin criterio alguno.

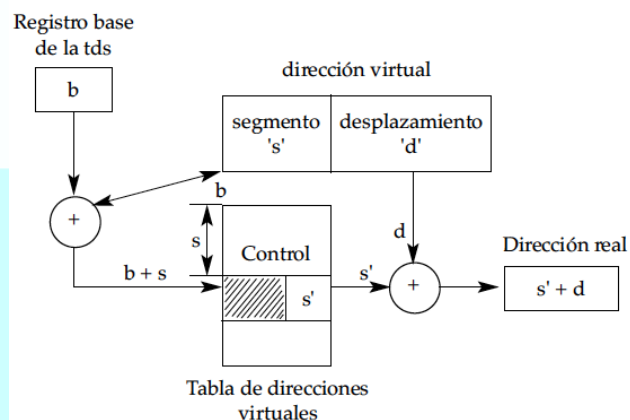
SEGMENTACIÓN pura

La segmentación aprovecha el hecho de que los programas se dividen en partes lógicas, como son la parte de datos, de código y de pila (stack).

Entonces, un programa es un conjunto de componentes lógicos de tamaño variable o un conjunto de segmentos, es decir, el espacio lógico de direcciones se considera como un conjunto de segmentos, cada uno definido por un identificador, y consistente de un punto de inicio y el tamaño asignado.

La segmentación asigna particiones de memoria a cada segmento de un programa y busca como objetivo el hacer fácil el compartir segmentos (por ejemplo, las librerías compartidas) y el intercambio entre memoria y los medios de almacenamiento secundario.

Como problema de la segmentación pura, se destaca la **fragmentación externa**, es decir, disponer de zonas libres de memoria principal que por su tamaño inferior a un segmento no pueden ser asignadas.



Prácticamente la traducción de direcciones virtuales es igual que la llevada a cabo en la paginación pura, tomando en consideración que el tamaño de los bloques a controlar por la tabla de direcciones son **variables**, por lo cual, cada entrada en dicha tabla debe contener la longitud de cada segmento a controlar.

Otra vez se cuenta con un registro base que contiene la dirección del comienzo de la tabla de segmentos. La dirección virtual se compone de un número de segmento (s) y un desplazamiento (d) para ubicar un byte (o palabra) dentro de dicho segmento. Es importante que el desplazamiento no sea mayor que el tamaño del segmento, lo cual se controla simplemente chequeando que ese valor sea mayor que la dirección del inicio del segmento y menor que el inicio sumado al tamaño.

Una vez dada una dirección virtual $v = (s, d)$, se realiza la operación $b + s$ para hallar el registro (o entrada de la tabla de segmentos) que contiene la dirección de inicio del segmento en la memoria real, denotado por s' . Conociendo la dirección de inicio en memoria real s' sólo resta

encontrar el byte o palabra deseada, lo cual se hace sumándole a s' el valor del desplazamiento, de modo que la dirección real $r = s' + d$.

Cada entrada en la **TABLA de SEGMENTOS** tiene campos que indican la longitud, los permisos, la presencia o ausencia y dirección de inicio en memoria real del segmento.

Esquemas COMBINADOS

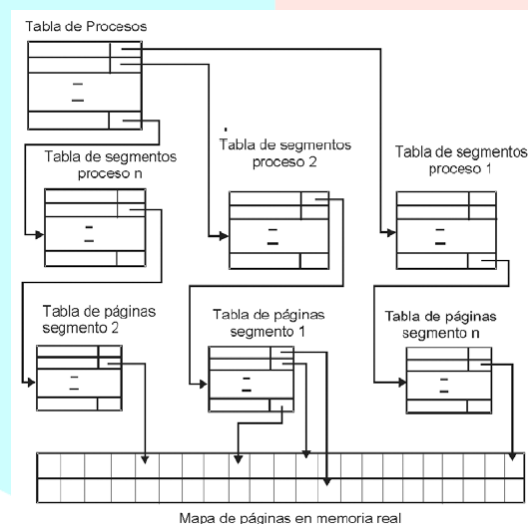
La paginación y la segmentación puras son métodos de manejo de memoria bastante efectivos, aunque la mayoría de los sistemas operativos modernos implantan esquemas combinados, es decir, combinan la paginación y la segmentación.

La idea de combinar estos esquemas se debe a que de esta forma se aprovechan los conceptos de la división lógica de los programas (segmentos) con la granularidad de las páginas.

De esta forma, un proceso estará repartido en la memoria real en pequeñas unidades (páginas) cuya liga son los segmentos. También es factible así el compartir segmentos a medida que las partes necesitadas de los mismos se van referenciando (páginas).

Para comprender este esquema, nuevamente se verá cómo se traduce una dirección virtual en una posición de memoria real. Para la paginación y segmentación puras se puede decir que el direccionamiento es bidimensional porque se necesitan dos valores para hallar la dirección real. Para el caso combinado, se puede decir que se tiene un direccionamiento tridimensional.

En la figura siguiente se muestran las partes relevantes para lograr la traducción de direcciones.



El sistema debe contar con una **tabla de procesos** (TP). Cada entrada de la tabla tiene un número de proceso y una dirección a una tabla de segmentos. Es decir, cada proceso tiene una tabla de segmentos.

Cuando un proceso hace alguna referencia a memoria, se consulta la TP para encontrar la tabla de segmentos de ese proceso.

En cada **TABLA de SEGMENTOS DE PROCESO** (TSP) se tienen los números de los segmentos que componen ese proceso. Por cada segmento se tiene una dirección a una tabla de páginas.

Cada **TABLA de PÁGINAS DEL SEGMENTO** (TPS) tiene las direcciones de las páginas que componen un solo segmento.

Así, por ejemplo, el segmento A puede estar formado por las páginas reales 2, 4, 6 y 8. El segmento B puede estar compuesto de las páginas 1, 3, 7 y 10.

Para traducir una dirección virtual $v = (s,p,d)$ donde s es el segmento, p es la página y d el desplazamiento en la página se hace lo siguiente: primero se ubica de qué proceso es el segmento y se localiza la tabla de segmentos de ese proceso en la TP. Con s como índice se encuentra el registro en la tabla de segmentos de ese proceso y en esa entrada está la dirección de la tabla de páginas que componen el segmento. Una vez en la tabla de páginas se usa el valor p como índice para encontrar la dirección de la página en memoria real. Una vez en esa dirección de memoria real se encuentra el byte (o palabra) requerido por medio del valor de d .

En este esquema pueden generarse dos tipos de fallos: el **fallo de página** y el **fallo de segmento**. Cuando se hace referencia a una dirección y el segmento que la contiene no está en memoria principal, se provoca un fallo por falta de segmento y lo que se hace es traerlo del medio de almacenamiento secundario y crearle una tabla de páginas. Una vez cargado el segmento se necesita localizar la página correspondiente, pero ésta no existe en memoria, por lo cual se provoca un fallo de página y se carga de disco y finalmente se puede ya traer la dirección deseada por medio del desplazamiento de la dirección virtual.

Las estrategias para cargar páginas (o segmentos) a la memoria más comunes son:

- **Carga de páginas POR DEMANDA**: consiste en que las páginas solamente son llevadas a memoria si se referencia.
- **Carga de páginas ANTICIPADA**: consiste en tratar de adivinar qué páginas serán solicitadas en el futuro inmediato y cargarlas de antemano, para que cuando se pidan ya no ocurran fallos de página. Para ello, se aprovecha el fenómeno de localidad y se cargan por anticipado que contienen direcciones contiguas a la dirección que se acaba de referenciar.

7. ADMINISTRACIÓN DEL SISTEMA DE E/S

El **sistema de entrada/salida (E/S)** es la parte del sistema operativo encargada de la gestión de los dispositivos de E/S. Actúa como interfaz entre los dispositivos y los usuarios, de manera que los archivos y dispositivos se traten de una manera uniforme y puedan ser manipulados por medio de instrucciones de alto nivel.

Los dispositivos de entrada salida se dividen, en general, en dos tipos:

- **Dispositivos orientados a BLOQUE**: tienen la propiedad de que se pueden direccionar, esto es, el programador puede escribir o leer cualquier bloque del dispositivo realizando primero una operación de posicionamiento sobre él. Los dispositivos más comunes orientados a bloque son los discos duros, la memoria o discos compactos.
- **Dispositivos orientados a CARÁCTER**: son aquellos que trabajan con secuencias de bytes sin importar su longitud ni ninguna agrupación en especial. No son dispositivos direccionables. Ejemplos de estos dispositivos son el teclado, la pantalla y las impresoras.

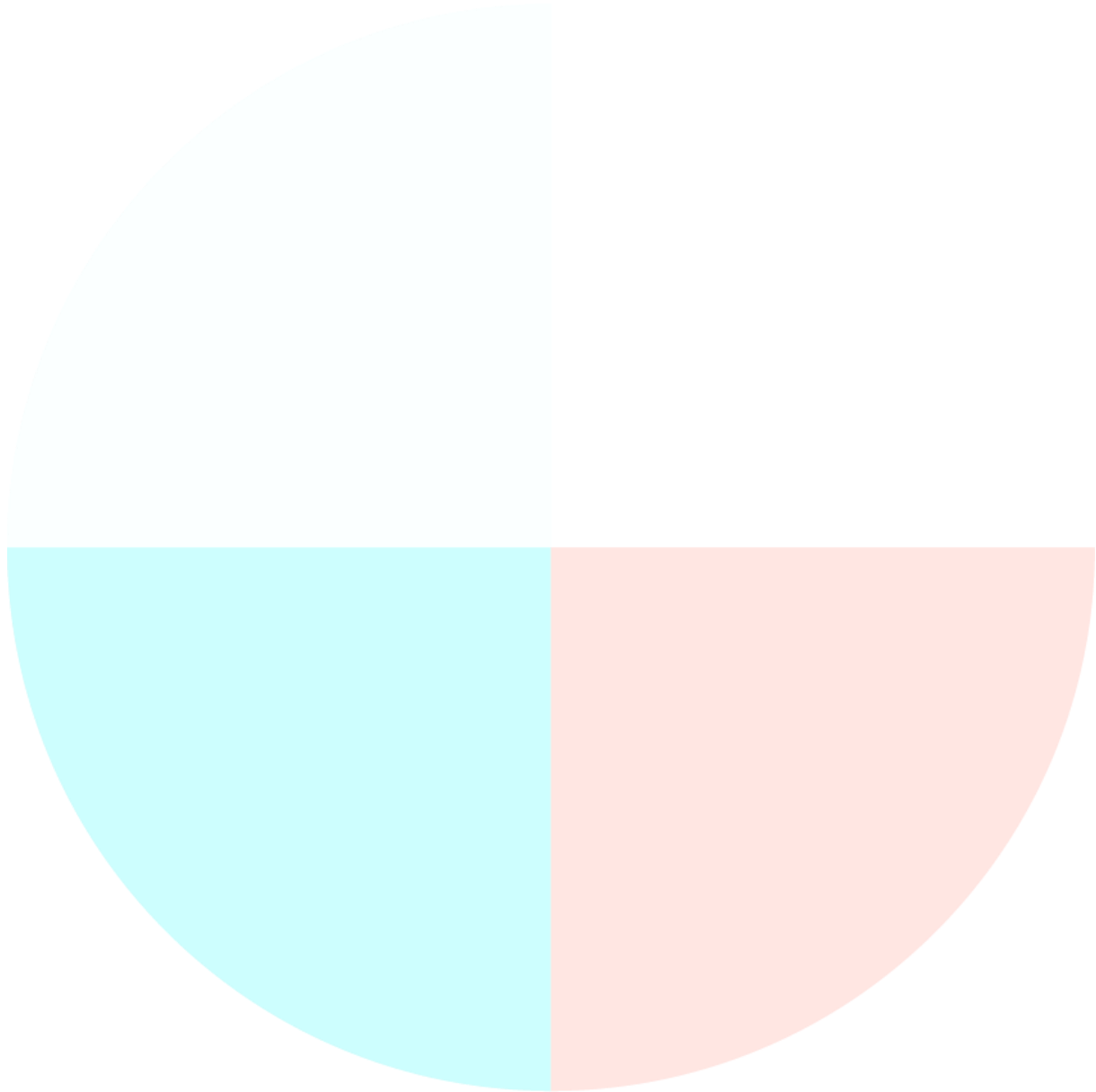
Los **controladores de dispositivos** (también llamados adaptadores de dispositivos) son la parte electrónica de los periféricos, los cuales pueden tener la forma de una tarjeta o un circuito impreso integrado en la placa base.

Conviene destacar el **acceso directo a memoria (DMA)** cuyo objetivo es liberar a la CPU de la carga de atender a algunos controladores de dispositivos, es decir, permite la transferencia de datos entre un periférico y la memoria sin intervención de la CPU.

Los **principios de software** en la E/S se resumen en cuatro puntos:

- **Manejadores de interrupciones**: el primer objetivo referente a los manejadores de interrupciones consiste en que el usuario no debe darse cuenta de los manejos de bajo nivel para los casos en que el dispositivo esté ocupado y se deba suspender el proceso o sincronizar algunas tareas. Desde el punto de vista del usuario, el sistema simplemente tardó más o menos en responder a su petición.
- **Manejadores de dispositivos**: el sistema debe proporcionar los manejadores de dispositivos necesarios para los periféricos, así como ocultar las peculiaridades del manejo interno de cada uno de ellos, tales como el formato de la información, los medios mecánicos, los niveles de voltaje y otros. Por ejemplo, si el sistema tiene varios tipos diferentes de discos duros, para el usuario o programador las diferencias técnicas existentes entre ellos no le deben importar. Los manejadores le deben ofrecer el mismo conjunto de rutinas para leer y escribir datos.
- **Software independiente del dispositivo**: este es un nivel superior de independencia que el ofrecido por los manejadores de dispositivos. Aquí el sistema operativo debe ser capaz, en la medida de lo posible, de ofrecer un conjunto de utilidades para acceder a periféricos o programarlos de una manera consistente. Por ejemplo, que para todos los dispositivos orientados a bloque se disponga de una llamada para decidir si se desea usar buffers o no, o para posicionarse en ellos.

- **Software para usuarios**: la mayoría de las rutinas de E/S trabajan en modo privilegiado o son llamadas al sistema, que se ligan a los programas del usuario formando parte de sus aplicaciones y que no le dejan ninguna flexibilidad en cuanto a la apariencia de los datos. Existen otras librerías en donde el usuario sí tiene poder de decisión (printf en el lenguaje C).



8. ADMINISTRACIÓN DE ARCHIVOS

Un **sistema de archivos** (File System) es una estructura de directorios con algún tipo de organización el cual nos permite almacenar, crear y borrar archivos en diferentes formatos.

ALGORITMOS de planificación de peticiones

El esquema básico de un sistema de almacenamiento es: las superficies de almacenamiento se dividen en círculos concéntricos "pistas", las cuales se dividen en "sectores". A la unión lógica de varias pistas paralelas se le denomina "cilindro".

Los algoritmos de planificación de peticiones de lectura y escritura a discos se encargan de tratar dichas peticiones:

- **Primero en llegar, primero en ser servido (FCFS o FIFO)**: las peticiones son encoladas de acuerdo al orden en que llegaron y de esa misma forma se van leyendo o escribiendo las mismas.
- **Primero el más cercano a la posición actual (SSTF)**: en este algoritmo las peticiones se ordenan de acuerdo a la posición actual de la cabeza lectora, sirviendo primero a aquellas peticiones más cercanas y reduciendo, así, el movimiento del brazo, lo cual constituye la ventaja principal de este algoritmo. Puede producir inanición.
- **Por exploración (algoritmo del elevador o SCAN)**: en este algoritmo el brazo se está moviendo en todo momento desde el perímetro del disco hacia su centro y viceversa, resolviendo las peticiones que existan en la dirección que tenga en turno.
- **Por exploración circular (C-SCAN)**: es una variación del algoritmo anterior, con la única diferencia que al llegar a la parte central, el brazo regresa al exterior sin resolver ninguna petición, lo cual proporciona un tiempo de respuesta más cercano al promedio para todas las peticiones.

ASIGNACIÓN del espacio de almacenamiento

El subsistema de archivos se debe encargar de localizar espacio libre en los medios de almacenamiento para guardar archivos y para después borrarlos, renombrarlos o aumentar su tamaño. Para ello, se vale de ubicaciones especiales que contienen la lista de archivos creados y por cada archivo una serie de direcciones que apuntan a su contenido. Esas localidades especiales se llaman directorios. Para asignarle espacio a los archivos existen tres criterios generales:

- **Asignación CONTIGUA**: cada directorio contiene los nombres de archivos y la dirección del bloque inicial de cada archivo, así como el tamaño total de cada uno. Si el archivo es borrado y luego creado otro más pequeño, quedarán huecos inútiles entre archivos útiles, lo cual se denomina **fragmentación externa**.

- **Asignación ENCADENADA**: con este criterio los directorios contienen los nombres de archivos y por cada uno de ellos la dirección del bloque inicial que compone al archivo. Con este criterio no es necesario que los bloques estén contiguos y no existe la fragmentación externa, pero en cada eslabón de la cadena se desperdicia espacio con las direcciones mismas. En otras palabras, lo que se crea en el disco es una lista encadenada.
- **Asignación con índices (INDEXADA o inodo)**: en este esquema se guarda en el directorio un bloque de índices para cada archivo, con apuntadores hacia todos sus bloques constituyentes, de manera que el acceso directo se agiliza notablemente, a cambio de sacrificar varios bloques para almacenar dichos apuntadores. Cuando se quiere leer un archivo o cualquiera de sus partes, se hacen dos accesos: uno al bloque de índices y otro a la dirección deseada. Éste es un esquema excelente para archivos grandes pero no para pequeños, para los que, la relación entre bloques destinados a índices respecto de los destinados para datos supone un coste inasumible.

Métodos de acceso en los sistemas de archivos

Los métodos de acceso se refieren a las capacidades que el subsistema de archivos proporciona para acceder a datos dentro de los directorios y medios de almacenamiento en general. Se presentan tres métodos de acceso:

- **Acceso SECUENCIAL**: es el método más lento y consiste en recorrer los componentes de un archivo uno a uno hasta llegar al registro deseado. Se necesita que el orden lógico de los registros sea igual al orden físico en el medio de almacenamiento. Este tipo de acceso se usa comúnmente en cintas y cartuchos.
- **Acceso DIRECTO**: permite acceder a cualquier sector o registro inmediatamente, por medio de llamadas al sistema como la de seek. Este tipo de acceso es rápido y se usa comúnmente en discos duros y discos o archivos manejados en memoria principal.
- **Acceso DIRECTO INDEXADO**: este tipo de acceso es útil para grandes volúmenes de información o datos. Consiste en que cada archivo tiene una tabla de apuntadores, donde cada apuntador va a la dirección de un bloque de índices, lo cual permite que el archivo se expanda a través de un espacio enorme. Consume una cantidad importante de recursos en las tablas de índices pero es muy rápido.

Operaciones

Las operaciones básicas que deben soportar los sistemas de archivos son: crear, borrar, abrir, cerrar, leer, escribir, concatenar, localizar, leer atributos, modificar atributos y renombrar.

9. SISTEMAS DE FICHEROS EXISTENTES

Los sistemas de ficheros los podemos clasificar en sistemas:

- **De disco**: los archivos se almacenan en una unidad disco.
- **De red**: los archivos se almacenan en una red.

Los sistemas de ficheros **de disco** más importantes son:

- **ext2, ext3 y ext4** (Linux).
- **HFS, HFS+ y APFS** (macOS).
- **FAT, FAT16** (MS-DOS) y **FAT32** (Windows y memorias USB).
- **extFAT** (Microsoft).
- **NTFS** (Windows).
- **ReFS** (Windows).
- **ReiserFS** (Linux).
- **Reiser4** (Linux).
- **JFS y JFS2** (IBM).
- **UFS y UFS2** (Unix).
- **VFS**.
- **XFS** (Linux).
- **ISO 9660** (discos ópticos).
- **F2FS** (memorias flash).

Respecto a los sistemas de ficheros **de red** destacan:

- **NFS** (Network File System): protocolo de sistemas de ficheros en red que permite que clientes puedan acceder a ficheros a través de la red como si el sistema de archivos fuese local. NFSv4 es la última versión.
- **AFS** (Andrew File System): sistema de archivos distribuido.
- **CIFS** (Common Internet File System): protocolo desarrollado por Microsoft que permite compartir archivos e impresoras entre nodos de una red (evolución de SMB -Server Message Block-). En sistemas Unix, macOS y GNU Linux existe la implementación libre Samba.

ext2 (Second Extended File System)

Es un sistema de archivos diseñado para Linux. Su estructura se basa en el concepto de inodo. Reemplazado por ext3.

ext3 (Third Extended File System)

Es un sistema de archivos diseñado para Linux. Su estructura se basa en el concepto de inodo. La principal diferencia con ext2 es que implemente el journaling (registro por diario), es decir, trabaja como un gestor transaccional. Sustituido por ext4.

ext4 (Fourth Extended File System)

Es un sistema de archivos transaccional implementado para Linux. Utiliza el concepto de **extends** (conjunto de bloques físicos contiguos) para trabajar con ficheros de gran tamaño.

Características más relevantes:

- Tamaño máximo archivo: 16 TB.
- Tamaño máximo volumen: 1 EB.
- Admite el cifrado.
- Compatible con versiones anteriores.

HFS (Hierarchical File System)

Es un sistema de archivos diseñado para macOS. Su estructura está basada en árboles B*. Sustituido por HFS+.

Características más relevantes:

- Tamaño máximo archivo: 2 GB.
- Tamaño máximo volumen: 2 TB.

HFS+

Es un sistema de archivos diseñado para macOS. En Linux se puede trabajar con este sistema de ficheros sin problema, sin embargo, en Windows solo se puede leer el contenido de las unidades formateadas con HFS+ pero no escribir.

Características más relevantes:

- Tamaño máximo archivo: 8 EB.
- Tamaño máximo volumen: 8 EB.
- Uso de sectores de 512 Bytes.
- Admite el cifrado.

APFS (Apple File System)

Es el sistema de archivos actual de Apple para máquinas con macOS 10.13 o posterior, tvOs y watchOS. Su diseño está basado en inodos. Está optimizado para unidades flash y discos SSD.

Características:

- Tamaño máximo archivo: 2 EB.
- Tamaño máximo volumen: 7,5 EB.
- Encriptación nativa.
- Soporta compresión.
- Permite instantáneas.

FAT (File Allocation Table)

Tabla de asignación de archivos es un sistema de archivos desarrollado inicialmente para MS-DOS así como para las primeras versiones domésticas de Windows. Actualmente FAT se conoce como FAT12, debido a que las direcciones de bloque contienen 12 bits.

Es un sistema de archivos sencillo, lo que lo hace muy popular en la gestión de discos y otras memorias secundarias externas y que sea aceptado por todos los sistemas operativos existentes (por lo que también es utilizado como mecanismo de intercambio de ficheros entre sistemas operativos distintos y coexistentes en el mismo equipo).

FAT tiene bastantes desventajas:

- Tiende a dejar fragmentos de los ficheros tras borrarlos, lo que con el tiempo ralentiza las operaciones de lectura y escritura.
- No es redundante a fallos (como fallos eléctricos).
- Fue diseñado para ficheros de tamaño reducido.
- No soporta permisos de seguridad.

Características:

- Tamaño máximo archivo: 32 MB.
- Tamaño máximo volumen: 32 MB.
- No admite compresión ni cifrado.

FAT16

Evolución de FAT. Utiliza direcciones de bloque de 16 bits.

Características:

- Tamaño máximo archivo: 2 GB.
- Tamaño máximo volumen: 2 GB.
- No admite compresión ni cifrado.

FAT32

Aparece con Windows 95. Utiliza direcciones de bloque de 32 bits.

Características:

- Tamaño máximo archivo: 4 GB.
- Tamaño máximo volumen: 10 GB.
- No admite compresión ni cifrado.
- Compatible con Windows, macOS y Linux.

exFAT (Extended File Allocation Table)

Es el sucesor de FAT32, propiedad de Microsoft. De gran uso en memorias flash y tarjetas SD. Es posible utilizar este sistema de ficheros en Linux.

Características:

- Tamaño máximo archivo: 16 EB.
- Tamaño máximo volumen: 64 ZB.
- Soporta permisos de seguridad.
- No soporta compresión y cifrado.
- Compatible con Windows, macOS y Linux.

NTFS (New Technology File System)

Desarrollado por Microsoft y presentado con el sistema operativo Windows NT.

En macOS se puede leer el contenido de las unidades formateadas con NTFS pero no escribir.

Características:

- Tamaño máximo archivo: 256 TB.
- Tamaño máximo volumen: 256 TB.
- Utiliza estructuras de datos avanzadas (árboles B) para optimizar el rendimiento
- Soporta permisos de seguridad (listas ACL).
- Soporta compresión y cifrado.

ReFS (Resilient File System)

Introducido con Windows Server 2012, ofrece gran rendimiento y escalabilidad. Destaca por la disponibilidad de datos e integridad de los datos.

Características:

- Tamaño máximo archivo: 35 PB.

- Tamaño máximo volumen: 35 PB.
- Longitud máxima nombre de archivo: 255 caracteres Unicode.

ReiserFS

Es un sistema de archivos transaccional implementado para Linux.

Características:

- Tamaño máximo archivo: 8 TB.
- Tamaño máximo volumen: 16 TB.
- Journaling.
- Redimensionamiento del sistema de ficheros montado.
- Utiliza estructuras de datos avanzadas (árboles B+).
- No soporta compresión y cifrado.

Reiser4

Evolución del sistema de ficheros ReiserFS. Implementado también para Linux.

Características:

- Tamaño máximo archivo: 8 TB.
- Journaling.
- Soporte eficiente para ficheros de tamaño pequeño.
- Redimensionamiento del sistema de ficheros montado.
- Utiliza estructuras de datos avanzadas (árboles B* balanceados).
- Soporta compresión y cifrado.

JFS (Journaling File System) y JFS2

Desarrollado por IBM para su sistema operativo AIX, compatible con Linux. JFS2 está pensado para kernel de 64 bits y archivos de tamaño mayor.

Características:

- Tamaño máximo archivo: 64 GB (JFS) y 16 TB (JFS2).
- Tamaño máximo volumen: 1 TB (JFS) y 32 TB (JFS2).
- Journaling.
- Soporta permisos de seguridad (listas ACL).
- Utiliza estructuras de datos avanzadas en JFS2 (árboles B).
- Permite instantáneas (JFS2).

- Soporta compresión (JFS).
- Número de inodos variable (JFS2).

UFS (Unix File System) y UFS2

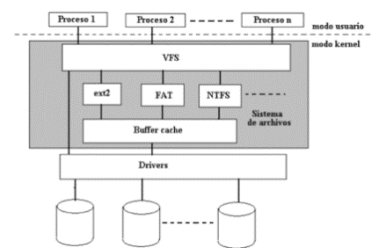
UFS deriva del Berkeley Fast File System (FFS). UFS2 es una extensión de UFS.

Características:

- Tamaño máximo archivo: 8 ZB.
- Tamaño máximo volumen: 8 ZB.

VFS (Virtual File System)

Es una interfaz que se sitúa justo encima de uno o varios sistemas de ficheros concretos de sistemas operativos diferentes (Windows, Linux, Mac OS). Es una capa de abstracción que permite a los procesos clientes realizar peticiones a VFS, el cual trasladará la operación solicitada al sistema de ficheros concreto y devolverá el resultado al proceso solicitante.



XFS

Desarrollado inicialmente por Silicon Graphics, posteriormente fue portado a Linux. Sistema de 64 bits.

Características:

- Tamaño máximo archivo: 8 EB.
- Tamaño máximo volumen: 8 EB.
- Journaling.
- Alta escalabilidad.
- Utiliza estructuras de datos avanzadas (árboles B+).
- No soporta compresión y cifrado.

ISO 9660 (discos ópticos)

Este estándar define un sistema de ficheros para discos ópticos, admisible en todas las plataformas. Como extensiones a este sistema de archivos tenemos: Joliet (uso Unicode), Romeo, Rock Ridge (permisos modo Linux), El Torito (disco autoarrancable) y extensiones de Apple.

F2FS (Flash-Friendly File System)

Creado por Samsung para el núcleo de Linux teniéndose en cuenta las características de los dispositivos basados en memorias flash NAND (SSD, SD, MMC).

10. ARQUITECTURAS DE SISTEMAS OPERATIVOS

Las arquitecturas actuales son:

- 32 bits (x86).
- 64 bits (x64).

La principal diferencia entre ambas es la cantidad de memoria que se puede direccionar. El número de bits hace referencia al ancho del bus de direcciones.

Por tanto, un procesador de 32 bits puede direccionar hasta un máximo de **4 GB** de RAM. En cambio, un procesador de 64 bits, es capaz de direccionar hasta un máximo de **16 EB**.

Dadas estas arquitecturas, la implementación de los sistemas operativos se condiciona para poder trabajar con una u otra arquitectura, incluso con ambas a la vez.

1. Interoperabilidad de procesos en Windows

Ejecución de aplicaciones Win32 en Windows de 64 bits

Se pueden ejecutar aplicaciones basadas en Win32 en Windows de 64 bits mediante una capa de emulación.

WOW64 es el emulador x86 que permite que las aplicaciones basadas en Windows de 32 bits se ejecuten sin problemas en Windows de 64 bits.

Téngase en cuenta que en Windows de 64 bits un proceso de 32 bits no puede cargar un archivo DLL de 64 bits ni viceversa (un proceso de 64 bits no puede cargar una DLL de 32 bits). Sin embargo, Windows de 64 bits admite llamadas a procedimientos remotos (**RPC**) entre procesos de 64 y 32 bits (tanto en el mismo equipo como entre equipos).

11. SISTEMAS WINDOWS

1. Familia Windows: puestos de usuario

Windows es la familia de sistemas operativos para PC, móviles y servidores desarrollados por Microsoft.

La primera versión de Windows (1.0) fue dada a conocer en el año 1985 por Microsoft. Se trataba de una interfaz gráfica de usuario para el sistema operativo MS-DOS. En adelante, la empresa lanzó nuevas versiones, pasando por la 2.0 y la 3.0, la versión 95, 98, NT, 2000, XP Vista, 7, 8, 10 y 11 con una serie de aplicaciones y funciones mucho más personalizadas y avanzadas para el usuario.

Actualmente, se trata del sistema operativo más usado y vendido del mundo. No obstante, hoy encuentra una gran competencia con los sistemas operativos de Apple y Linux, un sistema de uso libre y código abierto. La adquisición de Windows incluye en general otras aplicaciones de uso frecuente.

A pesar de su popularidad, el sistema ha sido largamente criticado por otras empresas y usuarios que encuentran en el mismo no sólo una copia de otros programas como los diseñados por Apple sino también problemas en la usabilidad y en sus criterios de seguridad, los cuales lo hacen vulnerable a múltiples virus y programas maliciosos.

- **Windows NT (New Technology)**: sistema operativo orientado a estaciones de trabajo y servidores de red. Distribuido en dos versiones, Workstation para estaciones de trabajo y Server usado como servidor. Como características destacan alto rendimiento, seguridad, fiabilidad, multitarea y multiproceso, sistema de archivos NTFS y tolerancia a fallos. La estructura del sistema operativo es de tipo microkernel, destacando la capa de abstracción de hardware (HAL) que se sitúa por encima del hardware físico.
- **Windows 2000**: sucesor de Windows NT, es un sistema operativo empresarial para ejecutar servidores de red o de archivo.
- **Windows XP (eXPerience)**: está basado en la arquitectura y núcleo de Windows NT, siendo el primer sistema operativo de Microsoft con esta característica a nivel doméstico. Se ha distribuido bajo dos ediciones, Home y Professional.
- **Windows Vista**: sucede a Windows XP bajo el nombre en clave "Longhorn". Introduce como características destacables Windows Media Center y Windows Aero (la nueva interfaz de usuario).
- **Windows 7**: es el sucesor de Windows Vista. Ofrece soporte para discos duros virtuales, mejora el rendimiento en procesadores multinúcleo, proporciona soporte para sistemas con múltiples tarjetas gráficas y mejora el rendimiento de arranque del sistema. Las versiones de escritorio, tanto para arquitecturas de 32 bits como de 64 bits, se distribuyen bajo las ediciones Home Basic, Home Premium, Professional y Ultimate.

- **Windows 8:** sistema operativo para su uso en PC, tanto de escritorio como de negocio, portátiles, tabletas y servidores. Como cambio principal se encuentra la desaparición del Menú Inicio. Características: soporte para procesadores ARM y adaptación de la interfaz de usuario para uso en pantallas táctiles. La nueva interfaz de usuario es denominada **Modern UI**. Incluye, además, la app OneDrive.
- **Windows 10:** incorpora las siguientes novedades: BitLocker, Microsoft Cortana, Microsoft Edge, Windows Hello, Vista de Tareas (para gestionar escritorios virtuales) y virtualización de aplicaciones (App-V). La nueva interfaz de usuario **Continuum** ofrece dos modos, uno para pantallas táctiles y otro para ratón y teclado. El Menú de Inicio se recupera tras la desaparición en Windows 8. Se crea el programa Windows Insider para testar nuevas funcionalidades.

Se basa en la unificación de plataformas introduciendo una arquitectura de aplicaciones universales, que pueden ser diseñadas para ejecutarse en todas las familias de productos de Microsoft con un código casi idéntico (incluyendo PCs, tabletas, teléfonos inteligentes, sistemas embebidos, Xbox One y otras).

También se introduce la Vista de Tareas, un sistema de escritorio virtual, el navegador web Microsoft Edge y otras aplicaciones nuevas o actualizadas, un soporte integrado para iniciar sesión a través de huella digital o reconocimiento facial llamado Windows Hello.

- **Windows 11:** es el sistema operativo actual de Microsoft. Requiere **TPM** (Módulo de Plataforma Segura) que es un procesador criptográfico seguro. Novedades: HDR automático, grupos de escritorios (escritorios virtuales), Windows Information Protection (WIP), DirectStorage (tiempos de carga más rápidos), soporte para aplicaciones Android gracias al Subsistema de Windows para Android, panel Windows Widgets. La interfaz de usuario se denomina **Fluent Design**.

Incorpora con respecto a Windows 10: grupos de escritorios, Microsoft Teams, DirectStorage, uso de Widgets.

Las ediciones de este sistema operativo son:

- Windows 11 Home.
- Windows 11 Pro.
- Windows 11 Empresas.
- Windows 11 SE (Educación).
- Windows 11 IoT Enterprise.
- Windows 11 en modo S: versión optimizada para seguridad y rendimiento. Solo permite apps de Microsoft Store y requiere Microsoft Edge para navegar.

AppLocker

Introducido en Windows 7 permite especificar mediante reglas para permitir/denegar los usuarios o grupos de usuarios que pueden ejecutar determinadas aplicaciones basándose en identidades únicas de archivos.

BitLocker

Cifrado de unidad BitLocker es una característica de protección de datos que se integra en el sistema operativo (Windows 10 y 11) y soluciona las amenazas de robo o exposición de datos de equipos perdidos, sustraídos o retirados inadecuadamente.

Windows Defender

Windows Defender es un antivirus incluido en las últimas versiones de Windows (8.1, 10, 11) y ayuda a proteger el PC contra virus y malware. Además, hace las funciones de un firewall.

Para versiones más antiguas está disponible el software Microsoft Security Essentials, que ofrece protección contra malware, virus, spyware y otro software malintencionado.

Microsoft Cortana

Cortana es un asistente digital de Microsoft (disponible desde Windows 10) que permite reconocer la voz natural y responder a preguntas utilizando el motor de búsqueda de Bing.

Permite crear recordatorios, realizar seguimientos, enviar correos electrónicos, administrar el calendario, crear y administrar listas, reproducir música, buscar información y abrir cualquier aplicación del sistema.

Microsoft Edge

Es el navegador web creado por Microsoft e incorporado en Windows 10, reemplazando por tanto a Internet Explorer. Está basado en el motor Chromium.

Windows Hello

Funcionalidad incluida en Windows 10 que permite iniciar sesión mediante reconocimiento facial, huella dactilar o un PIN.

App-V

Microsoft Application Virtualization (App-V) para Windows 10 ofrece a los usuarios aplicaciones Win32 como aplicaciones virtuales. Las aplicaciones virtuales se instalan en servidores administrados de forma centralizada y se envían a los usuarios como un servicio en tiempo real y según sea necesario. Los usuarios inician aplicaciones virtuales desde puntos de acceso e interactúan como si estuvieran instaladas de forma local.

Windows Insider

El programa Windows Insider permite instalar y usar las últimas compilaciones de Windows Insider Preview y así poder testar las nuevas funcionalidades.

WSL (Windows Subsystem for Linux)

Capa de compatibilidad desarrollada por Microsoft (disponible ya la versión 2 de WSL) que simula un kernel de Linux que permite ejecutar aplicaciones propias. En definitiva, posibilita la integración de Linux en Windows 10.

Windows Information Protection (WIP)

Característica que permite proteger los datos de trabajo al cifrarlos y separarlos de los datos personales.

2. Active Directory

Un **directorio** es una estructura jerárquica que almacena información acerca de los objetos de la red.

Un **servicio de directorio**, como Active Directory Domain Services (AD DS), proporciona los métodos para almacenar los datos de directorio y hacer que estos datos estén disponibles para los usuarios y administradores de la red. Es una estructura basada en el servicio de directorio LDAP (Lightweight Directory Access Protocol).

Por ejemplo, AD DS almacena información acerca de las cuentas de usuario, como nombres, contraseñas, números de teléfono, etc., y permite que otros usuarios autorizados de la misma red tengan acceso a esta información.

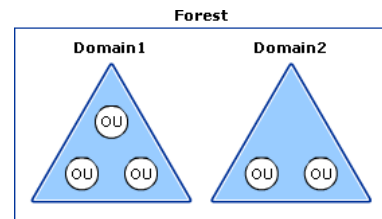
Active Directory almacena información acerca de los objetos de la red y facilita la búsqueda y el uso de esta información para los administradores y los usuarios. Active Directory usa un almacén de datos estructurado como base para una organización jerárquica lógica de información de directorio.

Este almacén de datos, también conocido como directorio, contiene información sobre los objetos de Active Directory. Estos objetos suelen incluir recursos compartidos como servidores, volúmenes, impresoras y las cuentas de equipo y usuario de red.

La seguridad se integra con Active Directory a través de la autenticación de inicio de sesión y el control de acceso a los objetos del directorio. Con un único inicio de sesión de red, los administradores pueden administrar los datos de directorio y la organización a través de su red, y los usuarios de red autorizados pueden tener acceso a los recursos en cualquier parte de la red. La administración basada en directiva facilita la administración de incluso las redes más complejas.

Los **dominios** dividen el directorio en secciones más pequeñas dentro de un solo bosque. Los nombres de dominio siguen la sintaxis del sistema DNS.

Los **bosques** son los límites de seguridad y el nivel más alto de la estructura lógica. Se pueden estructurar para proporcionar autonomía y aislamiento de datos y servicios en una organización de manera que puedan reflejar las identidades del sitio y del grupo y eliminar las dependencias de la topología física.



Las **unidades organizativas** permiten agrupar recursos, como cuentas o de usuario o PCs, en un dominio para fines de administración como una única unidad, lo que hace que sea más fácil aplicar la Política de grupo o delegar el control a los administradores.

La siguiente figura ilustra las relaciones de unidades organizativas, dominios y bosques en la arquitectura de estructura lógica.

Por fin, para que los usuarios de un dominio accedan a recursos de otro dominio hay que definir una **relación de confianza** entre dominios. Un sistema tipo Unix puede participar en una red Microsoft, actuando incluso como controlador de dominio. Para ello hay que instalar y configurar Samba en un sistema tipo Unix.

Las **Directivas de Grupo** (en adelante GPO) permiten implementar configuraciones específicas para uno o varios usuarios y/o equipos. Nos centraremos en cómo se debe hacer la gestión correcta de GPO en Active Directory de Microsoft Windows.

Para la configuración de GPO que sólo afecten a un usuario o equipo local se puede utilizar el editor de directivas locales **gpedit.msc**. En nuestro caso accederemos en el entorno de Servicios de Dominio de Active Directory, con la consola de administración **gpmc.msc**.

Las GPO permiten administrar objetos de usuarios y equipos, aplicando la más restrictiva en caso de existir más de una política. Se puede usar una GPO para casi cualquier cosa, como indicar qué usuario o grupo tiene acceso a una unidad de disco, o limitar el tamaño máximo que puede tener un archivo.

Las GPO se pueden diferenciar dependiendo del objeto al que configuran y se pueden entender en distintos niveles:

- **Equipo Local:** tan solo se aplican en el equipo que las tiene asignadas independientemente del dominio al que pertenezca.
- **Sitio:** se aplican a los equipos y/o usuarios de un sitio, independientemente del dominio.
- **Dominio:** se aplican a todos los equipos y/o usuarios de un dominio.
- **Unidad Organizativa (OU):** se aplican únicamente a los equipos y/o usuarios que pertenecen a la OU.

Dentro de la configuración de directiva se puede acceder a: *Configuración de equipo* y *Configuración de usuario*.

3. Línea de comandos

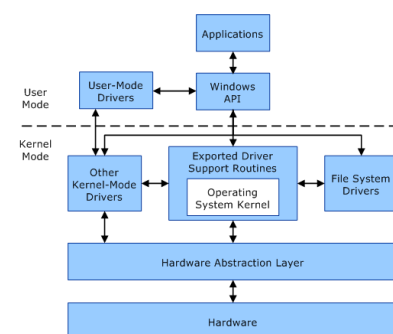
Los comandos de administración básica DOS habituales son los que se presentan a continuación:

Comandos DOS	
arp	Muestra o modifica la tabla ARP
ipconfig	Configuración IP en modo texto
ftp / tftp	Cliente FTP / TFTP
hostname	Muestra el nombre del host
finger	Informa de los usuarios que ejecutan el servicio finger
lpq	Muestra el estado de una cola de impresión remota (lpd)
lpr	Envía a imprimir a impresora remota
nbtstat	Muestra estadísticas NetBIOS
nslookup	Consultas a un servicio DNS
ping	Eco ICMP a host remoto
rcp	Copia archivos del equipo local a uno remoto que ejecute el servicio RCP
rexec / rsh	Ejecuta un comando en un host remoto con el servicio REXEC (RSH) habilitado
Comandos NET (Comandos DOS para red)	
netstat	Muestra estadísticas de red, conexiones y puertos abiertos
net computer	Agrega o elimina máquinas a un dominio
net start	Inicia un servicio. Sin parámetros, muestra los servicios en ejecución
net accounts	Gestiona parámetros de cuentas como la longitud mínima de la contraseña
net config	Gestiona la configuración de servicios
net print	Gestiona trabajos y colas de impresión
net pause	Suspende la ejecución de un servicio
net view	Muestra el listado de recursos que se están compartiendo en un servidor
net continue	Reanuda un servicio suspendido con net pause
net time	Sincroniza el reloj del sistema con el de un host remoto o dominio
net group	Agrega, muestra o modifica grupos globales
net localgroup	Agrega, muestra o modifica grupos locales
net name	Añade o elimina un alias
tracert	Realiza una traza a un host remoto
net send	Envía mensajes a otros usuarios
net stop	Detiene un servicio
net session	Muestra o desconecta sesiones del equipo. Sin parámetros, muestra las sesiones establecidas actualmente
net file	Desbloquea un archivo para otros usuarios. Sin parámetros, muestra los archivos compartidos abiertos
net use	Conexión a recurso remoto. El recurso remoto se mapea en una unidad de red. Ejemplo: net use X: \\servidor\recurso
net user	Crea y modifica cuentas de usuario. Sin parámetros, muestra las cuentas del sistema
net share	Gestiona recursos compartidos
net statistics	Estadísticas del servicio local

4. Modos de funcionamiento

Un procesador de un equipo que ejecuta Windows funciona en dos modos diferentes:

- **Modo de USUARIO:** se ejecuta el código de usuario, tienen acceso limitado a los recursos del sistema. Al iniciar una aplicación en modo de usuario, Windows crea un proceso para ella. Este proceso proporciona a la aplicación un espacio de direcciones virtuales privado y una tabla de identificadores privados. Dado que el espacio de direcciones virtuales de cada aplicación es privado, una aplicación no puede modificar los datos de otra aplicación. Cada aplicación se ejecuta de forma aislada, no afectando si se bloquea a otras aplicaciones ni al sistema operativo.
- **Modo KERNEL:** se ejecutan las funciones del kernel, tiene acceso completo al hardware y a los recursos del sistema. El código del sistema operativo se ejecuta en este modo. Todo el código que se ejecuta en modo kernel comparte un único espacio de direcciones virtuales.



Todos los componentes del modo kernel están totalmente protegidos frente a las aplicaciones que se ejecutan en modo de usuario. Se pueden agrupar de la siguiente manera:

- **Ejecutivo:** contiene los componentes del sistema operativo base, como administración de memoria, administración de procesos y subprocesos, seguridad, E/S, comunicación entre procesos.
- **Núcleo (kernel):** esto realiza funciones de bajo nivel, como programación de subprocesos, distribución de interrupciones y excepciones, y sincronización de varios procesadores. También proporciona un conjunto de rutinas y objetos básicos utilizados por el Ejecutivo.
- **Capa de abstracción de hardware (HAL, Hardware Abstraction Layer):** situado bajo los niveles kernel y ejecutivo, su propósito es presentar al resto del sistema operativo el hardware ocultando los detalles específicos sobre la versión del procesador, el conjunto de chips de soporte y otras variaciones de la configuración.

Nota: los sistemas operativos con una arquitectura microkernel implementan algunas de las funciones del sistema operativo como procesos de usuario.

5. Métodos de autenticación en Windows

Sistemas de autenticación:

- **Kerberos:** sistema de autenticación utilizado en los servicios de directorio.
- **Windows Hello:** servicio de inicio de sesión mediante biometría o PIN.
- **Microsoft Passport:** servicio de inicio de sesión único que permite a los usuarios tener una cuenta única que permite autenticarse en distintos dispositivos, sistemas operativos (desde W8), sitios web...
- **NTLM:** conjunto de protocolos de autenticación de usuarios y equipos basados en un mecanismo de desafío/respuesta que le demuestra a un servidor o a un controlador de dominio que un usuario conoce la contraseña asociada con una cuenta.

6. Registro de Windows

El registro de Windows es una base de datos jerárquica centralizada dispuesta para almacenar la información de configuración del sistema para usuarios, aplicaciones y dispositivos. La información del registro es la referencia que usa el sistema operativo continuamente, como puedan ser perfiles de usuario, aplicaciones instaladas, tipos de documentos que gestiona cada aplicación, elementos hardware del sistema, etc.

El registro reemplaza a la mayoría de archivos con extensión .ini basados en texto usados en versiones anteriores de Windows.

Una sección del registro es un grupo de claves, subclaves y valores que cuentan con archivos auxiliares con copias de seguridad de sus datos. Los archivos auxiliares de cada sección excepto HKEY_CURRENT_USER suelen estar en la carpeta %SystemRoot%\System32\Config.

Los tipos de valores más importantes usados en el registro para almacenar la información se resumen en la siguiente tabla.

NOMBRE	TIPO DE DATOS	DESCRIPCIÓN
Valor binario	REG_BINARY	Datos binarios sin formato. La mayoría de la información sobre componentes de hardware se almacena en forma de datos binarios y se muestra en formato hexadecimal en el Editor del Registro.
Valor DWORD	REG_DWORD	Datos representados por un número de 4 bytes de longitud (un valor entero de 32 bits). Muchos parámetros de controladores de dispositivo y servicios son de este tipo y se muestran en el Editor del Registro en formato binario, hexadecimal o decimal. DWORD_LITTLE_ENDIAN (el byte menos significativo está en la dirección inferior) y REG_DWORD_BIG_ENDIAN (el byte menos significativo está en la dirección superior) son valores relacionados.
Valor alfanumérico expandible	REG_EXPAND_SZ	Cadena de datos de longitud variable. Este tipo de datos incluye variables que se resuelven cuando un programa o servicio utiliza los datos.
Valor de cadena múltiple	REG_MULTI_SZ	Cadena múltiple. Valores que contienen listas o valores múltiples; este es el formato cuya lectura resulta más sencilla. Las entradas aparecen separadas por espacios, comas u otros signos de puntuación.
Valor de cadena	REG_SZ	Cadena de texto de longitud fija.
Valor binario	REG_FULL_RESOURCE_DESCRIPTOR	Serie de matrices anidadas diseñada para almacenar una lista de recursos utilizados por un dispositivo físico de hardware. El sistema detecta y escribe estos datos en el árbol \HardwareDescription que se muestra en el Editor del Registro en formato hexadecimal como valor binario.
Ninguna	REG_NONE	Datos sin ningún tipo en particular. El sistema o una aplicación escribe estos datos en el Registro y los muestra en el Editor del Registro en formato hexadecimal como un valor binario.
Vínculo	REG_LINK	Cadena Unicode que da nombre a un vínculo simbólico.
Valor QWORD	REG_QWORD	Datos representados por un número entero de 64 bytes. Estos datos se muestran en el Editor del Registro como un valor binario (incorporados en Windows 2000).

CARPETA O CLAVE PREDEFINIDA	DESCRIPCIÓN
HKEY_CURRENT_USER	Contiene la raíz de la información de configuración del usuario que ha iniciado sesión. Las carpetas del usuario, los colores de la pantalla y la configuración del Panel de control se almacenan aquí. Esta información está asociada al perfil del usuario. Esta clave a veces aparece abreviada como "HKCU".
HKEY_USERS	Contiene todos los perfiles de usuario cargados activamente en el equipo. HKEY_CURRENT_USER es una subclave de HKEY_USERS. HKEY_USERS puede aparecer abreviada como "HKU".
HKEY_LOCAL_MACHINE	Contiene información de configuración específica del equipo (para cualquier usuario). Esta clave a veces aparece abreviada como "HKLM".
HKEY_CLASSES_ROOT	Es una subclave de HKEY_LOCAL_MACHINE\Software. La información que se almacena aquí garantiza que cuando abra un archivo con el Explorador de Windows se abrirá el programa correcto. Esta clave a veces aparece abreviada como "HKCR". La clave HKEY_LOCAL_MACHINE\Software\Classes contiene la configuración predeterminada que se puede aplicar a todos los usuarios del equipo local. La clave HKEY_CURRENT_USER\Software\Classes contiene la configuración que invalida la configuración predeterminada y que se aplica únicamente al usuario interactivo. La clave HKEY_CLASSES_ROOT proporciona una vista del Registro que combina la información de estos dos orígenes. HKEY_CLASSES_ROOT también proporciona una vista combinada para los programas diseñados para versiones anteriores de Windows. Para cambiar la configuración del usuario interactivo, se deben realizar los cambios en HKEY_CURRENT_USER\Software\Classes en lugar de en HKEY_CLASSES_ROOT. Para cambiar la configuración predeterminada, se deben realizar los cambios en HKEY_LOCAL_MACHINE\Software\Classes. Si escribe valores en una clave de HKEY_CLASSES_ROOT, el sistema almacena la información en HKEY_LOCAL_MACHINE\Software\Classes. Si escribe valores para una clave en HKEY_CLASSES_ROOT y la clave ya existe en HKEY_CURRENT_USER\Software\Classes, el sistema almacenará la información ahí, en lugar de en HKLM\Software\Classes.
HKEY_CURRENT_CONFIG	Contiene información acerca del perfil de hardware que utiliza el equipo local cuando se inicia el sistema.

7. Microsoft Azure

Azure es un conjunto de servicios de computación en la nube enfocado a empresas.

La **informática en la nube** (cloud computing) es el suministro de servicios informáticos (incluidos servidores, almacenamiento, bases de datos, redes, software, análisis e inteligencia)

a través de Internet ("la nube"), cuyo objetivo es ofrecer una innovación más rápida, recursos flexibles y economías de escala. Lo habitual es pagar solo por los servicios en la nube utilizados, de tal forma que lo ayude a reducir los costos operativos, a ejecutar la infraestructura con más eficacia y a escalar a medida que cambian las necesidades de su negocio

No todas las nubes son iguales y un único tipo de informática en la nube no es apto para todo el mundo. Varios modelos, tipos y servicios distintos han evolucionado para ayudar a ofrecer la solución correcta según las necesidades.

En primer lugar, es necesario determinar el tipo de desarrollo de la nube o la arquitectura de informática en la nube en que se implementarán los servicios en la nube de una organización. Se pueden clasificar las estructuras de la nube en: nube pública, nube privada, nube comunitaria o nube híbrida.

- **Nube PÚBLICA:** la infraestructura de esta nube está mantenida y gestionada por terceras personas no vinculadas con la organización proporcionando recursos de forma abierta a entidades heterogéneas, sin más que un contrato con el mismo proveedor que controla dicha infraestructura.

Microsoft Azure es un ejemplo de nube pública. Con una nube pública, todo el hardware, software y demás componentes de la infraestructura subyacente son propiedad del proveedor de servicios en la nube, que también los administra. Usted obtiene acceso a estos servicios y administra su cuenta a través de un explorador web.

- **Nube PRIVADA** la infraestructura de esta nube o servicios provistos son completamente dedicados para un solo cliente que controla qué aplicaciones debe ejecutarse y dónde (infraestructura bajo demanda). Puede ser propiedad, ser administrado y operado por la organización, un tercero o alguna combinación de ellos, y puede existir dentro o fuera de las instalaciones.

La nube pública presenta flexibilidad de contratación y la nube privada, en la mayoría de los casos, exige determinados compromisos de consumo o permanencia. Una nube privada hace referencia a recursos informáticos en la nube que utiliza exclusivamente una empresa u organización.

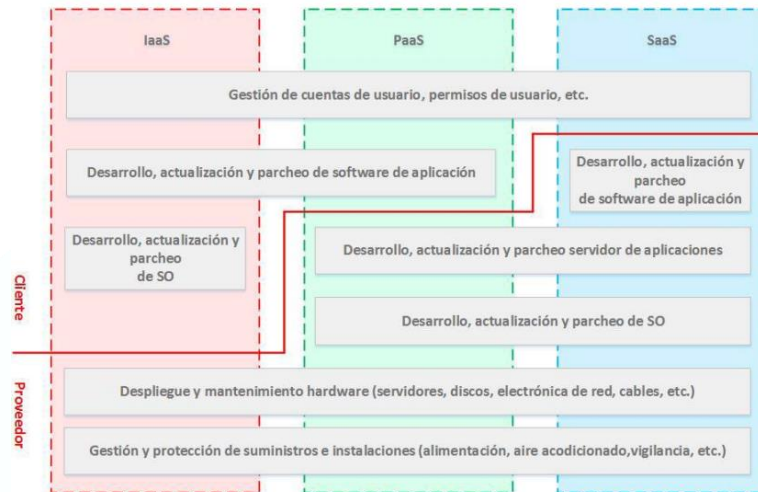
- **Nube HÍBRIDA:** los servicios se ofrecen de forma pública y privada. Un usuario es propietario de unas partes y comparte otras, aunque de una manera controlada.

Las nubes híbridas combinan nubes públicas y privadas, enlazadas mediante tecnología que permite compartir datos y aplicaciones entre ellas. Al permitir que los datos y las aplicaciones se desplacen entre las nubes privadas y públicas, una nube híbrida aporta a la organización mayor flexibilidad, más opciones de desarrollo y ayuda a optimizar la infraestructura, la seguridad y el cumplimiento existentes.

- **Nube COMUNITARIA:** la infraestructura de esta nube o servicios provistos son compartidos en comunidad cerrada por varias organizaciones relacionadas entre ellas y que comparten requisitos con la finalidad de servir a una función o propósito común (seguridad, política...).

La nube comunitaria puede ser propiedad, administrada y operada por una o más de las organizaciones de la comunidad, un tercero o alguna combinación de ellas.

La mayoría de los servicios de informática en la nube se engloban en 5 categorías generales:



- **INFRAESTRUCTURA como servicio (IaaS)**: se encarga de entregar una infraestructura al usuario, proporcionando recursos de procesamiento y almacenamiento a través de la red, sin ningún otro valor añadido (servicios de uso de almacenamiento, hardware, servidores y componentes de red).

El proveedor se encarga de la administración de la infraestructura y el cliente tiene el control sobre los sistemas operativos, almacenamiento y aplicaciones desplegadas, así como el control de los componentes de red.

Es la categoría más básica de servicios informáticos en la nube. Con IaaS, se alquila infraestructura de TI a un proveedor de servicios en la nube y se paga por uso.

- **PLATAFORMA como servicio (PaaS)**: se encarga de entregar una plataforma a la organización cliente. El cliente no administra ni controla la infraestructura, pero tiene el control sobre las aplicaciones instaladas y su configuración, y puede incluso instalar nuevas aplicaciones.

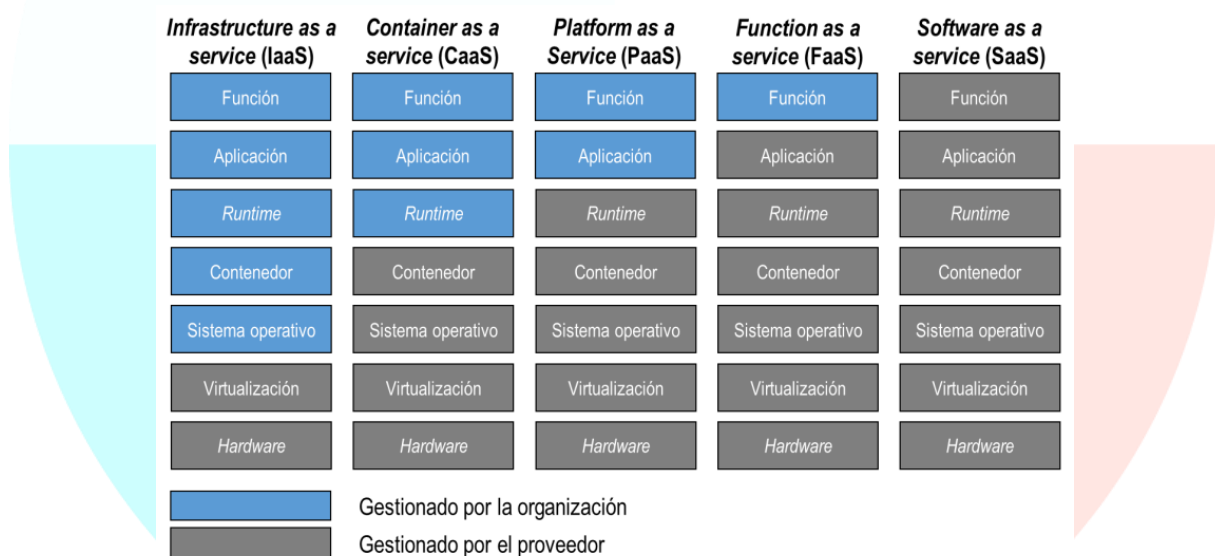
Se proporcionan herramientas y utilidades para desarrollar aplicaciones sobre la nube como bases de datos o entornos de programación en las que el usuario puede desarrollar sus propias soluciones.

PaaS está diseñado para facilitar a los desarrolladores la creación rápida de aplicaciones web o móviles, sin necesidad de preocuparse por la configuración o administración de la infraestructura de servidores subyacente, el almacenamiento, la red y las bases de datos necesarias para el desarrollo.

- **CONTENEDOR como servicio (CaaS)**: permiten gestionar e implementar las aplicaciones usando el aislamiento en contenedores en la nube.

- **FUNCIÓN como servicio (FaaS):** se centra en crear la funcionalidad de aplicaciones sin tener que dedicar constantemente el tiempo que conlleva tener que administrar los servidores y la infraestructura. Es lo que se conoce como “**INFORMÁTICA SIN SERVIDOR**”. El proveedor de servicios en la nube se encarga de la configuración, la gestión de la capacidad, la administración del servidor y las aplicaciones de software.
- **SOFTWARE como servicio (SaaS):** modelo de distribución de software en el que las aplicaciones están alojadas por un proveedor de servicio y puestas a disposición de los usuarios a través de red. El usuario encuentra en la nube las herramientas con las que puede implementar los procesos necesarios: aplicaciones ofimáticas, correo electrónico, etc... donde el cliente no administra ni controla la infraestructura en que se basa el servicio que utiliza.

Es un método de entrega de aplicaciones a través de Internet a petición y, normalmente, con una suscripción. Con SaaS, los proveedores de nube hospedan y administran las aplicaciones y la infraestructura subyacente, y se encargan del mantenimiento, como la aplicación de actualizaciones de software y revisiones de seguridad. Los usuarios se conectan a la aplicación a través de Internet, normalmente con un navegador web en su dispositivo móvil, tableta o PC.



- **CUALQUIER COSA como servicio (XaaS, Anything as a Service):** ofrecen cualquier tipo de elemento como un servicio. Destacan:
 - **DBaaS (Database as a Service).**
 - **BaaS (Backend as a Service).**
 - **STaaS (Storage as a Service):**
 - **DRaaS (Disaster Recovery as a Service).**
 - **AaaS (Analytics as a Service).**
 - **DaaS (Desktop as a Service).**

La informática en la nube supone un gran cambio respecto a la forma tradicional en la que las empresas veían los recursos de TI. Estos son los siete motivos habituales por los que las organizaciones están recurriendo a los servicios de informática en la nube:

- **Coste**: se elimina la inversión de capital que supone la adquisición de hardware y software, y la configuración y ejecución de centros de datos locales (bastidores de servidores, suministro eléctrico ininterrumpido para alimentación y refrigeración y expertos en TI para administrar la infraestructura).
- **Velocidad**: la mayoría de los servicios en la nube se proporcionan como autoservicio y a petición, de forma que incluso cantidades enormes de recursos informáticos se pueden aprovisionar en cuestión de minutos, lo que aporta a las empresas una gran flexibilidad y la relajación de no tener que preocuparse por planear la capacidad.
- **Escala global**: capacidad de escalar los recursos de forma elástica. En términos de nube, esto significa ofrecer la cantidad adecuada de recursos de TI (por ejemplo, mayor o menor capacidad de proceso, almacenamiento y ancho de banda) en el momento justo en el que se necesitan y desde la ubicación geográfica adecuada.
- **Rendimiento**: los mayores servicios informáticos en la nube se ejecutan en una red mundial de centros de datos seguros, que se actualizan periódicamente con el hardware más rápido y eficiente de última generación. Esto aporta varias ventajas en comparación con un único centro de datos corporativo, entre las que se incluyen una latencia de red menor para las aplicaciones y mayores economías de escala.
- **Seguridad**: muchos proveedores de nube ofrecen un conjunto completo de directivas, tecnologías y controles que refuerzan la situación general de seguridad, ayudando a proteger los datos, las aplicaciones y la infraestructura frente a posibles amenazas.
- **Productividad**: los CPDs suelen necesitar una cantidad importante de bastidores y apilamiento, lo que supone configuración de hardware, aplicación de revisiones de software y otras tareas de administración de TI que resultan tediosas y requieren mucho tiempo. La informática en la nube elimina la necesidad de muchas de estas tareas, de forma que los equipos de TI pueden dedicar su tiempo a lograr objetivos más importantes para su negocio.
- **Confiabilidad**: facilita y abarata la creación de copias de seguridad de los datos, la recuperación ante desastres y la continuidad empresarial, ya que los datos se pueden reflejar en varios sitios redundantes en la red del proveedor de servicios en la nube.

12. SISTEMAS UNIX Y LINUX

Unix es un sistema operativo portable, multitarea y multiusuario; desarrollado en 1969 por un grupo de empleados de los laboratorios Bell de AT&T, entre los que figuran Dennis Ritchie, Ken Thompson y Douglas McIlroy.

GNU/Linux es un conjunto de sistemas operativos libres multiplataforma, multiusuario y multitarea basados en Unix. El sistema es la combinación de varios proyectos, entre los cuales destacan GNU (acrónimo recursivo: GNU is Not Unix), encabezado por Richard Stallman y la Free Software Foundation junto con el núcleo o kernel Linux, programado por Linus Torvalds. Su desarrollo es uno de los ejemplos más prominentes de software libre: todo su código fuente puede ser utilizado, modificado y redistribuido libremente por cualquiera, bajo los términos de la licencia GPL y otra serie de licencias libres.

1. Distribuciones GNU/Linux

Se proponen a continuación las siguientes distribuciones Linux más importantes:

DISTRIBUCIÓN	BASADO EN	GESTOR DE PAQUETES	OBSERVACIONES
Debian		APT	
Kali Linux	Debian	dpkg	Auditoría y seguridad
Knoppix	Debian	APT	1ª distribución live
Q4OS	Debian/Ubuntu	APT	
Trisquel	Debian/Ubuntu	APT	
Ubuntu	Debian	APT	
Elementary OS	Ubuntu	APT	
Linux Mint	Ubuntu	APT	
Tuquito	Ubuntu	APT	
Kubuntu	Ubuntu		
Xubuntu	Ubuntu		
Zorin OS	Ubuntu		
Edubuntu	Ubuntu	APT	Entorno educativo
Lubuntu	Ubuntu	dpkg	
Peppermint OS	Debian		

Arch Linux		Pacman	
Manjaro	Arch Linux	Pacman	
Gentoo Linux		ebuild	
Ututo	Gentoo	Ebuild	
Red Hat Enterprise Linux (RHEL)		RPM	
CentOS	RHEL	YUM	
Fedora	Red Hat Linux	RPM, YUM	
Mandriva		RPM	
PCLinuxOS	Mandriva	RPM	
Puppy Linux			
openSUSE	SUSE Linux	YaST	

Un **gestor de paquetes** es una colección de herramientas que sirven para automatizar el proceso de instalación, actualización, configuración y eliminación de paquetes de software.

GNU

Las distribuciones libres del sistema GNU/Linux incluyen y ofrecen únicamente software libre. Rechazan aplicaciones que no sean libres, plataformas de programación que no sean libres, controladores que no sean libres, “blobs” de firmware que no sean libres, y cualquier otro tipo de software o documentación que no sea libre.

Distribuciones libre de GNU:

- Para equipos:
 - Dragora.
 - Dynebolic.
 - Guix System.
 - Hyperbola: basada en Arch.
 - Parabola: basada en Arch.
 - PureOS: basada en Debian.
 - Trisquel.
 - Ututo S.
- Para dispositivos con recursos limitados:
 - libreCMC.
 - ProteanOS.

GNU Hurd es el núcleo de un sistema operativo que forma parte del Proyecto GNU para reemplazar el núcleo de Unix. Posee las siguientes características:

- Es software libre, por lo que cualquiera puede usarlo, modificarlo y redistribuirlo bajo la licencia GPL.
- Está basado en la arquitectura microkernel.
- Cada parte del sistema está diseñada para ser modificada y extendida fácilmente.
- Tiene una estructura orientada a objetos que le permite evolucionar sin comprometer su diseño.
- Hurd está construido de una manera muy modular, la mayoría de los componentes que constituyen todo el núcleo se ejecutan como procesos de espacio de usuario separados.

2. Entornos de escritorio y shells

Las interfaces gráficas de usuario o también denominados entornos de escritorio disponibles en Linux son:

- **GNOME**: primer entorno más popular. Consumen menos recursos que KDE.
- **KDE**: segundo entorno de escritorio más popular. La última versión se denomina Plasma.
- **Cinnamon**: entorno con experiencia de usuario similar a sistemas Windows.
- **Unity**: entorno basado en GNOME creado por la empresa Canonical.
- **XFCE**: es un entorno de escritorio orientado a dispositivos con recursos limitados cuyo objetivo es ser rápido y usar pocos recursos del sistema.
- **MATE**: es un fork de GNOME.
- **LXDE** (Lightweight X11 Desktop Environment).
- **LXQt**: utiliza LXDE.

Listado de shells:

- Bourne Shell.
- BASH (Bourne-again Shell).
- Ksh.
- Z Shell.
- C Shell.
- TCsh.

3. Aspectos generales

Una partición de disco clásica contiene un sistema de archivos con la organización que se ilustra en la figura siguiente:



UNIX no usa el **bloque 0 (bloque de arranque)**, que a menudo contiene código para arrancar el ordenador. El **bloque 1** es el **superbloque**; contiene información crucial acerca de la organización del sistema de archivos, incluido el número de i-nodos, el número de bloques de disco y el principio de la lista de bloques de disco libres. La destrucción del superbloque hace que el sistema de archivos ya no pueda leerse.

Después del superbloque viene la **lista de i-nodos**, en el que cada entrada se asocia con un archivo (cada archivo tiene asociado un i-nodo). Un i-nodo contiene información de contabilidad (incluida toda la información devuelta por stat), así como información para localizar todos los bloques de disco que contienen los datos del archivo. Es decir, guarda la descripción del archivo: situación del archivo en disco, propietario, permisos de acceso, fechas...

Después de la lista de i-nodos están los **bloques de datos**. Aquí se almacenan todos los archivos y directorios.

Si un archivo o directorio consta de más de un bloque, los bloques no necesitan estar contiguos en el disco. De hecho, lo más probable es que los bloques de un archivo grande estén dispersos por todo el disco.

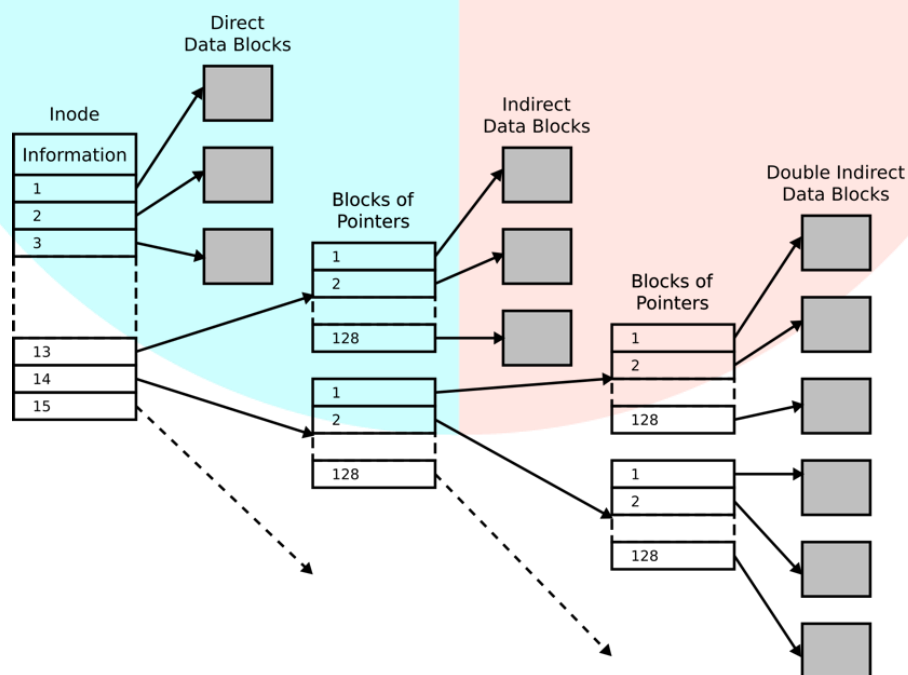
Por ejemplo, para buscar un archivo con un nombre de ruta absoluta (como /bin/local/archivo), se requieren los siguientes pasos:

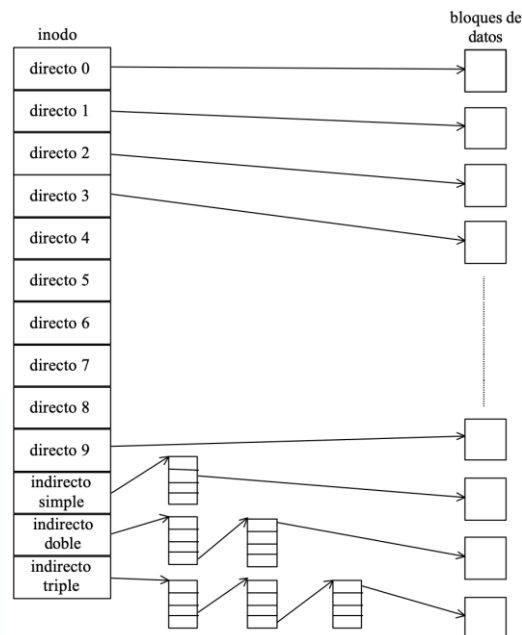
- En primer lugar, el sistema localiza el directorio raíz, que por lo general utiliza el i-nodo 2, en especial cuando el i-nodo 1 está reservado para el manejo de bloques defectuosos.
- Después busca la cadena «bin» en el directorio raíz, para obtener el número de i-nodo del directorio /bin.
- Después se obtiene este i-nodo y se extraen los bloques de disco del mismo, para poder leer el directorio /local y buscar la cadena «archivo».
- Una vez que se encuentra esta cadena, se puede obtener de ahí el número de nodo-i para el directorio /bin/local y se puede leer este i-nodo para localizar los bloques de directorio.
- Por último, se busca «archivo» y se encuentra el número de su i-nodo.

En la tabla siguiente tenemos los campos genéricos de un i-nodo:

CAMPO	DESCRIPCIÓN
UID	UID del propietario
GID	GID del propietario
Tipo	Tipo de archivo
Permisos	Permisos de lectura, escritura y ejecución para el propietario, el grupo y el resto de usuarios. El permiso de ejecución en directorios significa poder acceder o no a los archivos que contiene
Tiempo acceso	Fecha de último acceso al archivo
Tiempo modificación	Fecha de última modificación del archivo
Tiempo modificación i-nodo	Fecha de última modificación del i-nodo
Nº enlaces	Número de archivos que apuntan a ese i-nodo
Entradas	Direcciones de los bloques de datos en disco
Tamaño	Tamaño del archivo en bytes

Un i-nodo contiene, además de la información de seguridad y contabilidad necesarias, las direcciones en disco, siendo flexible para manejar archivos de tamaño pequeño y grande.





Direcciones de disco de un i-nodo:

- **Entradas DIRECTAS:** direcciones en disco de los primeros 10 (o 12) bloques del archivo.
- **1 entrada INDIRECTA SIMPLE:** en caso de archivos de más de 10 (o 12) bloques, un campo en el i-nodo contiene la dirección en disco de un bloque de indirección sencilla, este bloque contiene las direcciones en disco de más bloques del archivo.
- **1 entrada INDIRECTA DOBLE:** si no es suficiente, el siguiente campo en el i-nodo contiene la dirección en disco de un bloque de doble indirección, que contiene las direcciones en disco de más bloques de indirección sencilla, que a su vez contienen la dirección en disco de más bloques del archivo.
- **1 entrada INDIRECTA TRIPLE:** por último, si aún es preciso un archivo mayor, el último campo del i-nodo contiene la dirección de un bloque de triple indirección que, con la misma lógica de los casos anteriores, contiene las direcciones en disco de más bloques de doble indirección.

4. Modos de funcionamiento

En sistemas Unix/Linux son necesarios 2 modos de ejecución:

- **Modo USUARIO:** se ejecuta el código de usuario. El intento de ejecución de una instrucción privilegiada (son aquellas que potencialmente pueden dañar a otros procesos, como operaciones de E/S, actualización del reloj, desactivación de interrupciones, manipulación de la MMU...) en este modo produce una excepción.
- **Modo KERNEL:** se ejecutan las funciones del kernel. El código del sistema operativo se ejecuta en este modo. Por ello, se denomina kernel al código del sistema operativo que se ejecuta en este modo. Entre las funciones que se ejecutan en este modo podemos citar:

- Llamadas al sistema: los procesos de usuario solicitan servicios explícitamente a través de la interfaz de llamadas al sistema.
- Excepciones: situaciones excepcionales (división por cero, errores de direccionamiento...) causan excepciones hardware que requieren intervención del kernel.
- Interrupciones: los dispositivos periféricos interrumpen para notificar al kernel de diversos sucesos (terminación de E/S, cambio de estado...).

El cambio de modo usuario a modo kernel ocurre cuando se produce una interrupción, excepción o un trap. Las instrucciones hardware solo pueden ser ejecutadas en modo kernel. Es por ello que los servicios del sistema operativo se obtienen invocando un procedimiento del sistema a través de una llamada al sistema.

Nota: los sistemas operativos con una arquitectura microkernel implementan algunas de las funciones del sistema operativo como procesos de usuario.

5. Procesos

Las entidades activas principales en un sistema Unix/Linux son los procesos.

Cada proceso ejecuta un solo programa, y al principio tiene un solo hilo de control. En otras palabras, tiene un contador del programa que lleva la cuenta de la siguiente instrucción a ejecutar. Linux permite que un proceso cree hilos adicionales, una vez que empieza a ejecutarse.

Linux es un sistema de multiprogramación, por lo que puede haber varios procesos independientes en ejecución al mismo tiempo. De hecho, en las máquinas de un solo usuario, hay docenas de procesos ejecutándose en segundo plano. Así pues, podemos distinguir entre:

- Procesos interactivos.
- Procesos demonio: que se ejecutan en background. Estos procesos se inician mediante una secuencia de comandos de shell cuando se inicia el sistema. Uno de los demonios comunes es cron. Se despierta a intervalos de tiempo constantes (por ejemplo, una vez por minuto) para revisar si hay trabajo por hacer. De ser así, realiza las tareas pendientes. Después regresa a la inactividad hasta que sea hora de la siguiente verificación.

En Linux, los procesos se crean de una forma especialmente simple. La llamada al sistema `fork` crea una copia exacta del proceso original. El proceso que va a realizar la bifurcación es el proceso padre. Al nuevo proceso se le conoce como proceso hijo. El padre y el hijo tienen cada uno sus propias imágenes de memoria privadas. Si el padre cambia después una de sus variables, los cambios no son visibles para el hijo, y viceversa.

Los archivos abiertos se comparten entre el padre y el hijo. Es decir, si se abrió cierto archivo en el padre antes de `fork`, seguirá abierto tanto en el padre como en el hijo de ahí en adelante. Los cambios que realice cualquiera de los dos procesos será visible para el otro. El hecho de

que las imágenes de memoria, variables registros y todo lo demás sea idéntico en el padre y el hijo genera una pequeña dificultad: ¿Cómo saben los procesos cuál de ellos debe ejecutar el código del padre y cuál debe ejecutar el código del hijo? El secreto es que la llamada al sistema `fork` devuelve el valor 0 para el hijo y un valor distinto de cero, el PID (Process Identifier, Identificador de proceso) del hijo, al padre.

Los procesos se denominan de acuerdo con sus PIDs. Cuando se crea un proceso, el padre recibe el PID del hijo, como se mencionó antes. Si el hijo desea conocer su propio PID, la llamada al sistema `getpid` se lo proporciona. Los PIDs se utilizan en varias formas. Por ejemplo, cuando un proceso hijo termina, el padre recibe el PID del hijo que acaba de terminar. Esto puede ser importante debido a que un padre puede tener muchos hijos. Como los hijos también pueden tener hijos, un proceso original puede construir todo un árbol completo de hijos, nietos y más descendientes. Si un proceso termina y su padre todavía no lo espera, el proceso entra en un tipo de animación suspendida, conocida como estado zombie. Cuando por fin el padre lo está esperando, el proceso termina finalmente.

El comando `kill` nos permite enviar señales a los procesos que nos interese. En general, todos los programas se diseñan para que puedan recibir este tipo de señales. De este modo, según el tipo de señal recibido saben que deben realizar unas operaciones u otras. Hay muchos tipos diferentes de señales, que podemos consultar en la ayuda de `kill`, aunque las más utilizadas son las que nos sirven para obligar a un proceso a que termine o pause su ejecución. Con la señal `TERM` (`kill -15 PID`), le indicamos al proceso que queremos que termine, de modo que al recibir la señal deberá guardar todo lo necesario y acabar su ejecución. Si hay algún tipo de problema o el programa no está preparado para recibir este tipo de señal, podemos utilizar `kill -9 PID`, que automáticamente lo expulsa de la cola de ejecución.

`killall` sirve para referirnos al nombre de varios procesos a la vez en lugar de referenciarlos por su PID y, de este modo, enviarles una señal a todos a la vez.

Con el comando `skill` también podemos enviar señales a los procesos, pero con una sintaxis diferente. Por ejemplo, si queremos detener todas las ejecuciones de un determinado usuario, podríamos utilizar `skill -STOP -u nombreLogin`, con lo que todos los procesos de dicho usuario se pararían. Para reiniciarlos de nuevo, podríamos pasar la señal de `CONT`. Cuando estamos ejecutando algún programa en una consola y queremos pasarle la señal de `TERM`, podemos utilizar la combinación de teclas "Ctrl+C". Con "Ctrl+Z" podemos pausar un programa y revivirlo con `fg`.

En Linux, los procesos se pueden comunicar entre sí mediante el uso de formas de paso de mensajes como las tuberías (**pipes**) o envío de señales (signals). Es posible crear un canal entre dos procesos donde un proceso puede escribir un flujo de bytes para que el otro los lea. Estos canales se conocen como tuberías. La sincronización es posible debido a que, cuando un proceso trata de leer de una canalización vacía, se bloquea hasta que haya datos disponibles.

Las tuberías en línea del shell se implementan con símbolos de tubería "|". Cuando el shell ve una línea como: `sort | head`, crea dos procesos, `sort` y `head`, y establece una tubería de canalización entre ellos, de tal forma que la salida estándar de `sort` se conecta con la entrada estándar de `head`. De esta forma, todos los datos que escribe `sort` van directamente a `head`.

en vez de ir a un archivo. Si la tubería se llena, el sistema deja de ejecutar `sort` hasta que `head` haya eliminado ciertos datos de ella.

Hay varias llamadas al sistema relacionadas con las señales, que se utilizan de varias formas. Por ejemplo, si un usuario indica por accidente a un editor de texto que muestre el contenido completo de un archivo muy extenso y después se da cuenta del error, se necesita alguna forma de interrumpir al editor. La opción usual es que el usuario oprima cierta tecla especial (por ejemplo, `SUPR` o `CTRL-C`) que envíe una señal al editor. Éste atraparé la señal y detendrá la impresión.

Con el comando `jobs` podemos ver qué procesos están lanzados en modo `background` (pasándole el parámetro `-l` también podremos ver su `PID`).

Si quisiéramos pasar uno de estos procesos a modo **foreground** (como si lo hubiéramos lanzado desde la línea de comandos sin el carácter `"&"`), podemos utilizar el comando `fg` indicando el `PID` del proceso.

También existe `bg`, que nos envía un determinado proceso a modo *background*. Este último es útil cuando, por ejemplo, ejecutamos un programa en modo *foreground* y lo pausamos con `"Ctrl+Z"`. Si después ejecutamos `bg` indicándole su `PID`, el proceso continuará su ejecución en modo *background*.

El manejo de procesos en UNIX es por prioridad y Round Robin. En algunas versiones se maneja también un ajuste dinámico de la prioridad de acuerdo con el tiempo que los procesos han esperado y al tiempo que ya han usado la CPU. El sistema provee facilidades para contabilizar el uso de CPU por proceso y una pila común para todos los procesos cuando necesitan estarse ejecutando en modo privilegiado (cuando hicieron una llamada al sistema).

Cuando se inicia un proceso, se le asigna:

- Un identificador denominado **PID**.
- El identificador del proceso que lo lanzó **PPID** (proceso padre).
- El usuario propietario que lo lanzó **UID**.
- El grupo de pertenencia **GID**: lo que definirá el perfil de permisos de acceso a los que tendrá derecho.

Existe la posibilidad de alterar el usuario o grupo efectivo de permisos, durante la ejecución del proceso, mediante la llamada a **setuid** o **setgid**, siempre que se disponga de los permisos apropiados. También existe una bandera de permisos `setuid` asociada al archivo del programa que permite ejecutar éste, con los permisos del propietario del archivo en lugar de los del usuario que lo ejecuta.

Para tener compatibilidad con otros sistemas UNIX, Linux identifica a los procesos a través del Identificador de proceso (`PID`). Todo proceso es referenciado por un descriptor de proceso, el cual contiene los atributos del proceso, así como la información que permite gestionarlo.

El kernel organiza todos los procesos en una lista doblemente enlazada de estructuras de tareas denominada **tabla de procesos**.

Además de utilizarlo para recorrer las listas enlazadas y acceder a los descriptores de procesos, el PID se puede asignar a la dirección de la estructura de tarea o proceso, y se puede acceder a la información del proceso de inmediato (que contiene varios campos). Los más importantes se describen a continuación:

- **Parámetros de planificación**: prioridad del proceso, cantidad de tiempo de la CPU que consumió recientemente, tiempo que ha estado inactivo recientemente. En conjunto, estos parámetros se utilizan para determinar qué proceso se debe ejecutar a continuación.
- **Imagen de memoria**: apuntadores a los segmentos de texto, datos y pila, o tablas de páginas. Si el segmento de texto es compartido, el apuntador de texto apunta a la tabla de texto compartida. Cuando el proceso no está en memoria, la información sobre cómo encontrar sus partes en el disco también se encuentra aquí.
- **Registros de la CPU**: cuando se llama a una interrupción trap en el kernel, los registros de la CPU se guardan aquí.
- **Estado de llamada al sistema**: información sobre la llamada al sistema actual, incluyendo sus parámetros y resultados.
- **Tabla de descriptores de archivos**: cuando se invoca una llamada en la que se involucra un descriptor de archivos, se utiliza ese descriptor como índice en esta tabla para localizar la estructura de datos dentro del núcleo (i-nodo) que corresponde a este archivo.
- **Contabilidad**: apuntador a una tabla que lleva la cuenta del tiempo de la CPU de usuario y de sistema que utilizó el proceso. Algunos sistemas también mantienen límites aquí en cuanto a la cantidad de tiempo de CPU que puede utilizar un proceso, el tamaño máximo de su pila, el número de marcos de página que puede consumir y otros elementos.
- **Pila del kernel**: una pila fija para que la parte del kernel del proceso la utilice.

ps

El comando **ps** es el que permite informar sobre el estado de los procesos. **ps** está basado en el sistema de archivos */proc*, es decir, lee directamente la información de los archivos que se encuentran en este directorio. Tiene una gran cantidad de opciones, incluso estas opciones varían dependiendo del estilo en que se use el comando. Estas variaciones sobre el uso de **ps** son las siguientes:

- Estilo UNIX, donde las opciones van precedidas por un guion.
- Estilo BSD, donde las opciones no llevan guion.
- Estilo GNU, donde se utilizan nombres de opciones largas y van precedidas por doble guion.

Sea cual sea el estilo utilizado, dependiendo de las opciones indicadas, varias columnas se mostrarán en el listado de procesos que resulte, estas **columnas** pueden ser entre muchas otras, las siguientes (y principales):

COLUMNA	DESCRIPCIÓN
p o PID	Process ID, número único identificación del proceso.
P o PPID	Parent Process ID, padre del proceso.
U o UID	User ID, usuario propietario del proceso.
t o TT o TTY	Terminal asociada al proceso, si no hay terminal aparece entonces un '?'. T o TIME
T o TIME	Tiempo de uso de CPU acumulado por el proceso.
c o CMD	El nombre del programa o comando que inició el proceso.
RSS	Resident Sise, tamaño de la parte residente en memoria en kilobytes.
SZ o SIZE	Tamaño virtual de la imagen del proceso.
NI	Nice, valor nice (prioridad) del proceso, un número positivo significa menos tiempo de procesador y negativo más tiempo (-19 a 19)
C o PCPU	Porcentaje de CPU utilizado por el proceso.
STIME	Starting Time, hora de inicio del proceso.
S o STAT	Status del proceso, estos pueden ser los siguientes: - R runnable, en ejecución. - S sleeping, proceso en ejecución pero sin actividad por el momento, o esperando por algún evento para continuar. - T sTopped, proceso detenido totalmente, pero puede ser reiniciado. - Z zombie, difunto, proceso que por alguna razón no terminó de manera correcta, no debe haber procesos zombies. - D uninterruptible sleep, son procesos generalmente asociados a acciones de IO del sistema. - X dead, muerto, proceso terminado pero que sigue apareciendo, igual que los Z no deberían verse nunca.

```
#> ps --help
```

```
***** simple selection ***** ***** selection by list *****
```

```
-A all processes          -C by command name
```

```
-N negate selection      -G by real group ID (supports names)
```

```
-a all w/ tty except session leaders -U by real user ID (supports names)
```

```
-d all except session leaders -g by session OR by effective group name
```

```
-e all processes        -p by process ID
```

```
T all processes on this terminal -s processes in the sessions given
```

```
a all w/ tty, including other users -t by tty
```

```
g OBSOLETE -- DO NOT USE -u by effective user ID (supports names)
```

```

r only running processes      U processes for specified users
x processes w/o controlling ttys  t by tty
***** output format ***** ***** long options *****
-o,o user-defined -f full      --Group --User --pid --cols --ppid
-j,j job control  s signal     --group --user --sid --rows --info
-O,O preloaded -o v virtual memory --cumulative --format --deselect
-l,l long          u user-oriented --sort --tty --forest --version
-F extra full  X registers    --heading --no-heading --context
***** misc options *****
-V,V show version  L list format codes f ASCII art forest
-m,m,-L,-T,H threads  S children in sum  -y change -l format
-M,Z security data  c true command name -c scheduling class
-w,w wide output    n numeric WCHAN,UID -H process hierarchy

```

```

$ ps -e      (-e muestra todos los procesos)
$ ps -ef     (-f muestra opciones completas)
$ ps -eF     (-F muestra opciones completas extra)
$ ps -eo user,pid,tty      (-o output personalizado, los campos separados por coma)
$ ps -eH     (muestra árbol de procesos)
$ ps axf     (lo mismo en formato BSD)
$ ps -ec     (el comando que se está ejecutando, sin la ruta, solo el nombre real)
$ ps -el     (muestra formato largo de varias columnas, muy práctico)
$ ps L      (no muestra procesos, lista todos los códigos de formatos)

```

kill

El comando `kill` sirve no solo para matar o terminar procesos sino principalmente para enviar señales (signals) a los procesos. La señal por default (cuando no se indica ninguna es terminar o matar el proceso), y la sintaxis es `kill PID`, siendo PID el número de ID del proceso. Así por ejemplo, es posible enviar una señal de STOP al proceso y se detendrá su ejecución, después cuando se quiera mandar una señal de CONTinuar y el proceso continuará desde donde se quedó.

killall

El comando `killall`, que funciona de manera similar a `kill`, pero con la diferencia de en vez de indicar un PID se indica el nombre del programa, lo que afectará a todos los procesos que tengan ese nombre. Así por ejemplo si se tienen varias instancias ejecutándose del proxy server squid, con `killall squid` eliminará todos los procesos que se estén ejecutando con el nombre 'squid'.

nice

Permite cambiar la prioridad de un proceso. Por defecto, todos los procesos tienen una prioridad igual ante el CPU que es de 0. Con **nice** es posible iniciar un programa (proceso) con la prioridad modificada, más alta o baja según se requiera. Las prioridades van de -20 (la más alta) a 19 la más baja. Solo root o el superusuario puede establecer prioridades negativas que son más altas. Con la opción -1 de **ps** es posible observar la columna NI que muestra este valor.

renice

Así como **nice** establece la prioridad de un proceso cuando se inicia su ejecución, **renice** permite alterarla en tiempo real, sin necesidad de detener el proceso.

nohup y &

Cuando se trata ejecutar procesos en background (segundo plano) se utiliza el comando **nohup** o el operador **&**. Aunque realizan una función similar, no son lo mismo.

nohup (no hangups) ejecuta un comando que se le pasa como argumento de forma que cuando se cierre la terminal, el proceso lanzado ignora la señal **SIGHUP** y continúa su ejecución hasta su finalización. La salida estándar es reenviada a un fichero denominado **nohup.out**.

Sin embargo, con **&** el proceso en ejecución finaliza al cerrar la terminal. Es posible evitarlo uniendo ambos comandos (**nohup comando &**).

top

Monitorización en tiempo real del estado de los procesos y de otras variantes del sistema es el programa llamado **top**, se ejecuta desde la línea de comandos, es interactivo y por defecto se actualiza a intervalos periódicos.

6. Estructura de directorios

El estándar de jerarquía del sistema de ficheros (**FHS**, Filesystem Hierarchy Standard) es una norma que define los directorios principales y sus contenidos para el sistema operativo GNU/Linux y otros sistemas de la familia Unix. La estructura de directorios es:

DIRECTORIO	DESCRIPCIÓN
/	Directorio raíz, inicio del sistema de archivos
/bin	Archivos ejecutables, comandos de usuario (ls, cp, cat, mkdir, rm...)
/boot	Archivos de arranque
/dev	Ficheros de dispositivos
/dosc	Punto de montaje para la partición DOS
/etc	Ficheros de configuración, scripts de arranque
/home	Directorios de inicio de los usuarios
/lib	Librerías del sistema
/lost+found	Archivos perdidos
/mnt	Punto de montaje de dispositivos externos
/media	Puntos de montaje para dispositivos de medios
/opt	Paquetes de aplicaciones estáticas
/proc	Archivos de control de procesos /proc/version: versión kernel /proc/cpuinfo: información CPU /proc/meminfo: uso memoria /proc/stat: estadísticas del sistema /proc/uptime: tiempo en segundos que el sistema lleva encendido desde que se inició /proc/swaps: ficheros swap /proc/mounts: sistemas de ficheros montados /proc/net/dev: adaptadores de red y estadísticas /proc/interrupts: interrupciones
/root	Directorio de inicio del superusuario root
/sbin	Archivos ejecutables de administración del sistema
/tmp	Ficheros temporales
/usr	Ejecutables, documentación, referencia
/var	Archivos de log

7. Comandos sistema de ficheros

Para listar el contenido de los directorios utilizamos el comando

ls [opciones] [fichero...]

Las opciones del comando ls son:

OP	DESCRIPCIÓN
-l	Muestra un listado en el formato largo, con información de permisos, número de enlaces asociados al archivo, usuario, grupo, tamaño y fecha de última modificación además del nombre.
-h	Imprime el tamaño de los archivos.
-d	Muestra solo el nombre del subdirectorio, sin entrar en él ni dar un listado del contenido.
-t	Muestra el listado ordenado por la fecha de última modificación.
-r	Cuando el listado se hace por orden temporal, los archivos más recientes van al principio. Si se indica la -r se invierte el orden, mostrando los más recientes al final.
-L	En los enlaces simbólicos, muestra los datos del archivo referenciado en vez de los del link.
-1	Muestra el listado en una sola columna. Sin la opción -l el listado se muestra en varias columnas, tantas como permita el ancho de la terminal (generalmente controlado con la variable de entorno \$COLUMNS).
-i	Muestra el número del i-nodo antes del nombre de archivo.
-m	Muestra los archivos en una línea y separados por comas.
-R	Muestra un listado recursivo. Lista primero los archivos del directorio en curso, luego los de los subdirectorios de éste, luego los de los subdirectorios contenidos en ellos (nietos) y así sucesivamente.
-s	Muestra delante del nombre del fichero el tamaño en kilobytes del mismo.
-a	Muestra los archivos ocultos.

Los ficheros se renombran o se mueven de directorio con el comando **mv** y se eliminan con el comando **rm**. Por otro lado, los directorios se crean con **mkdir** y se eliminan con **rmdir**.

Para referirnos a un archivo o a un directorio debemos indicar su ruta (path). Un ejemplo de ruta podría ser:

```
/home/pablo/documentos/informe.docx
```

Este tipo de rutas en las que se especifican todos los subdirectorios empezando desde el directorio raíz se denominan **rutas absolutas**.

Para no tener que escribir la ruta absoluta completa cada vez que queremos referirnos a un archivo o a un directorio se crearon los conceptos de directorio de trabajo y de ruta relativa.

El **directorio de trabajo** es una propiedad del terminal (del shell) en la que estamos trabajando. Siempre que estemos trabajando en una terminal tendremos asignado un directorio de trabajo. Por ejemplo, si nuestro usuario es *pablo* sería normal que al abrir un terminal nuestro directorio de trabajo fuese:

```
/home/pablo
```

El directorio de trabajo se utiliza para escribir rutas a archivos relativas al mismo. De este modo nos ahorramos escribir bastante. Imaginemos que Pablo tiene en su directorio un documento llamado *datos.txt*. La ruta absoluta sería:

```
/home/pablo/datos.txt
```

Mientras su directorio de trabajo sea */home/pablo* la ruta relativa sería simplemente:

```
datos.txt
```

Es decir, podemos escribir **rutas relativas** al directorio de trabajo, rutas que en vez de partir del directorio raíz parten desde el directorio de trabajo.

Es común referirse al directorio de trabajo de una terminal como a un lugar en el que nos encontramos mientras estamos trabajando en la terminal. Siempre que estemos en una terminal estaremos dentro de un directorio de trabajo.

Por ejemplo, cuando abrimos un nuevo terminal el directorio de trabajo se sitúa en */home/nombre_de_usuario*. Si ejecutamos el comando *ls*, el shell asumirá que queremos listar los archivos presentes en ese directorio y no en otro cualquiera. Existe un comando que nos informa sobre el **directorio de trabajo actual**:

pwd (Print Working Directory)

```
$ pwd  
/home/pablo
```

Si deseamos podemos modificar el directorio de trabajo “moviéndonos” a otro directorio, utilizamos el comando

cd (Change Directory)

```
$ cd documentos  
$ pwd  
/home/pablo/documentos
```

Podemos pedir información sobre el espacio ocupado por las distintas particiones que tenemos actualmente montadas usando el comando **df** (Disk Free)

```
$ df -h
```

El comando **du** (Disk Usage) informa sobre el espacio que ocupa un árbol de directorios:

```
$ du -h
```

Si sólo queremos obtener el resultado para el directorio que le hemos dado y no para sus subdirectorios podemos utilizar el parámetro **-s**.

Los métodos de COMPRESIÓN y DESCOMPRESIÓN de ficheros se exponen a continuación:

- **Archivos .tar.gz:**
 - Comprimir: tar -czvf empaquetado.tar.gz /carpeta/a/empaquetar/
 - Descomprimir: tar -xzf archivo.tar.gz
 - Listar contenido: tar -tzvf paquete.tar.gz
- **Archivos .tar:**
 - Empaquetar: tar -cvf paquete.tar /dir/a/comprimir/
 - Desempaquetar: tar -xvf paquete.tar
 - Listar contenido: tar -tvf paquete.tar
- **Archivos .gz:**
 - Comprimir: gzip -9 index.php
 - Descomprimir: gzip -d index.php.gz
- **Archivos .zip:**
 - Comprimir: zip archivo.zip carpeta
 - Descomprimir: unzip archivo.zip
 - Listar contenido: unzip -l archivo.zip

Las variables del entorno predefinidas por el sistema que ponemos acceder anteponiendo \$ al nombre de esta son:

- **HOME**: directorio personal.
- **LOGNAME**: contiene el nombre del usuario con el que nos conectamos.
- **HOSTNAME**: nombre de la máquina.
- **PATH**: lista de directorios donde buscar los programas.
- **SHELL**: intérprete de comandos por defecto.
- **TERM**: tipo de terminal.
- **USER**: nombre del usuario.

Para **buscar dentro de un archivo** utilizamos el comando `grep`.

grep [options] pattern [FILE]

OP	DESCRIPCIÓN
-i	La búsqueda no distingue entre mayúsculas y minúsculas.
-c	Muestra el número de coincidencias (líneas) con el patrón buscado.
-r	Habilita la búsqueda recursiva en el directorio actual.
-n	Muestra el número de línea de cada coincidencia con el patrón buscado.
-v	Se muestran las líneas que no coinciden con el patrón buscado

El carácter `^` representa el inicio de una línea.

El carácter `$` representa el final de una línea.

```
$ grep -i query file
```

Busca la palabra `query` en el archivo `file` sin distinción entre mayúsculas y minúsculas.

```
$ grep -c Autor libro.txt
```

Muestra el número total de líneas que contienen "Autor" en el fichero `libro.txt`.

```
$ grep '^a' libro.txt
```

Muestra las líneas que comienzan por "a" en el fichero `libro.txt`.

```
$ grep '$' libro.txt
```

Muestra las líneas que terminan en "." en el fichero `libro.txt`.

El comando más común utilizado para encontrar y filtrar archivos en Linux es a través del comando **find**. El diseño básico de este comando es el siguiente:

find directorio_comienzo [options] termino_búsqueda

CRITERIO	EFFECTO
-name patrón	Busca ficheros que coincidan con el patrón (pueden usarse comodines...).
-iname	Igual que name pero no distingue mayúsculas/minúsculas.
-regex	Igual que name pero usa REGEXPR.

-type tipo	Busca por tipo de fichero (f=archivo normal, d=directorio, l=enlace simbólico, c=dispositivos de caracteres, b=dispositivos de bloque).
-size [+/-]n[bck]	Busca por tamaño (tamaño igual, mayor o menor que n con b=bloques, c=bytes y k=KB, M=megabytes, G=gigabytes).
-perm [-]permisos -perm /[ugo=rwx]	Busca por permisos (sin - la búsqueda es exacta, con - que tengan al menos esos permisos). Busca por permisos según agrupación.
-user nombre	Busca por propietario.
-group	Busca por grupo.
-nouser, -nogroup	Busca ficheros con prop./grupo no válidos
-atime [+/-]n	Busca ficheros cuya fecha de acceso/lectura es anterior a (+) o es posterior a (-) n días.
-mtime [+/-]n	Busca ficheros modificados (n se mide en periodos de 24 horas).
-ctime [+/-]n	Busca ficheros según la fecha en que sus metadatos fueron actualizados.
-amin/-mmin/	Igual que -atime pero ahora n representa minutos
-cmin [+/-]n	Igual que -mtime pero ahora n representa minutos.
-newer file	Busca ficheros modificados más recientemente que file
-cnewer file	Ficheros con cambio de estado más reciente que la modificación de file.
-empty	Busca ficheros vacíos.
-exec CMD	Ejecuta una acción CMD para cada fichero que satisfaga las condiciones.

```
$ find / -name "[0-9]*"
```

Busca ficheros cuyo nombre comience por un dígito.

```
$ find / -name "[a-g]*"
```

Busca ficheros cuyo nombre comiencen entre a y g.

```
$ find / -type f -empty
```

Busca ficheros vacíos.

```
$ find / -perm /u=r
```

Busca ficheros de solo lectura para el propietario.

```
$ find / -perm 777
```

Busca ficheros con permisos 777.

```
$ find /home -user pablo -iname "*.txt"
```

Busca ficheros cuyo propietario es pablo y terminan en .txt.

```
$ find / -atime -50
```

Busca ficheros accedidos en los últimos 50 días.

```
$ find / -mtime +50 -mtime -100
```

Busca ficheros modificados hace más de 50 días y menos de 100 días.

```
$ find / -size +100M -exec rm -f {} \;
```

Borra los ficheros con un tamaño superior a 100MB.

```
$ find / -amin -60
```

Busca ficheros accedidos en la última hora.

```
$ find / -size 50M
```

Busca ficheros cuyo tamaño es 50MB.

```
$ find / -size +50M -size -100M
```

Busca ficheros cuyo tamaño es mayor a 50MB y menor a 100MB

xargs comando

La utilidad xargs (eXtended ARGuments) se usa para ejecutar comandos sobre la salida estándar.

```
find / -type f | xargs grep "texto"
```

Busca los ficheros que contienen texto.

ln [-s] archivo_apuntado nombre_enlace

Un enlace es una entrada a un mismo archivo físico desde ubicaciones diferentes, tanto si el enlace es físico como simbólico:

- Un **enlace simbólico** (ln -s) guarda la ruta del archivo referenciado y puede establecerse entre particiones diferentes, y no desaparece si elimina el archivo al que hace referencia.
- Un **enlace físico** solo se puede establecer dentro de una misma partición y para eliminar el archivo físico primero se deben eliminar todos los enlaces físicos que lo referencian.

Además, todos los enlaces físicos de un archivo comparten el propietario y los permisos de acceso.

```
$ ln -s /etc/passwd /home/usuario/claves
```

Crea un enlace simbólico claves que apunta al fichero passwd.

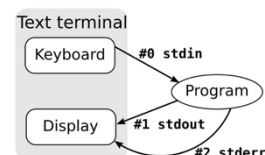
Por último, como editores de texto en línea de comandos tenemos **vi** y **nano**.

8. Flujos de información

En Linux, vamos a trabajar con el concepto de **descriptor de fichero**. Un descriptor se trata de un número entero que identifica una cierta operación con un fichero o dispositivo.

La **tabla de descriptores de fichero** se trata de una estructura de datos propia de cada proceso existente en el sistema. Por tanto, existe una tabla de descriptores de fichero por proceso. La tabla de descriptores de fichero tiene un número limitado de descriptores y, además, ya viene con tres descriptores abiertos de partida, que son los descriptores 0, 1 y 2:

- File Descriptor **0 STDIN**: dispositivo de entrada estándar, por defecto el teclado.
- File Descriptor **1 STDOUT**: dispositivo de salida estándar, por defecto la pantalla.
- File Descriptor **2 STDERR**: dispositivo de salida de mensajes de error estándar, por defecto la terminal o pantalla.



Tenemos las siguientes opciones:

SÍMBOLO	DESCRIPCIÓN
>	Redirecciona stdout hacia un archivo. Lo crea si no existe, si existe lo sobrescribe. ls -l > lista.txt (la salida del comando se envía a un archivo en vez de al terminal)
>>	Redirecciona stdout hacia un archivo. Lo crea si no existe, si existe concatena la salida al final de este. ps -ef >> procesos.txt (concatena al archivo procesos.txt la salida del comando).
<	Redirecciona stdin desde un archivo. El contenido de un archivo es la entrada o input del comando. mail user < texto.txt (el cuerpo del correo a enviar proviene desde un archivo, en vez del teclado).
2> 2>>	Redirecciona stderr hacia un archivo. Crea (>) o concatena (>>) la salida de errores a un archivo.
1>&2	Redirecciona stdout hacia donde stderr apunte.
2>&1	Redirecciona stderr hacia donde stdout apunte.

```
$ sort < lista_desordenada.txt > lista_ordenada.txt
```

```
$ cp archivo1 archivo2 2> /dev/null
```

/dev/null es un tipo especial de archivo que descarta toda la información que se escribe o se redirige a él. También se le conoce como "null device".

```
$ comando > todo.txt 2>&1
```

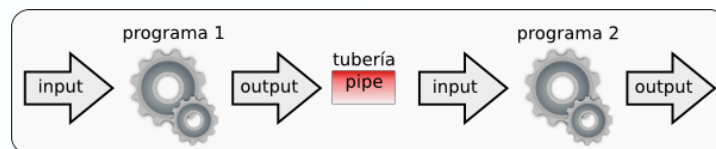
Redirecciona **stderr** hacía donde **stdout** apunte, y **stdout** apunta al archivo "todo.txt", por lo tanto, este archivo contendrá la salida del *comando* y los errores si los hubiera.

```
$ comando 2> errores.txt 1>&2
```

Redirecciona la salida del comando **stdout** hacia donde **stderr** apunte.

```
$ cp archivo1 archivo2 2> error.txt #redireccionamos stderr hacia un archivo
$ cp archivo1 archivo2 2>> error.txt #si se desea concatenar
```

Los flujos de información pueden ser redirigidos de tal manera que la salida de un programa sea la entrada de otro. Para ello, utilizamos una **tubería o pipe** (símbolo |).



```
$ ps -ef | grep -i bash
```

Lista los procesos y busca el comando bash.

9. Permisos de archivo y directorio

En Linux cualquier archivo o directorio tiene 3 niveles de permisos de acceso:

- Permisos del usuario propietario (UID).
- Permisos del grupo propietario (GID).
- Permisos del resto de usuarios. Comúnmente denominados "otros".

Partimos del siguiente ejemplo:

```
$ ls -l
-rwxrwxr-- pablo tic      2020-08-07 14:10 presentacion
-rw-rw-r-- pablo jefes    2020-09-16 16:36 informe
drwxr-xr-x pablo tic      2020-10-15 11:41 datos
```

Veamos por partes el listado, tomando como ejemplo la primera línea.

La 1ª columna (-rwxrwxr--) es el tipo de archivo y sus permisos, la 3ª columna (pablo) representa al propietario del archivo, la 4ª columna (tic) representa al grupo al que pertenece al archivo y las siguientes son el tamaño, la fecha y hora de última modificación y por último el nombre del archivo o directorio.

S₉S₈S₇S₆S₅S₄S₃S₂S₁S₀

s₉, primer carácter al extremo izquierdo, representa el tipo de archivo. Posibles valores son:

TIPO	DESCRIPCIÓN
-	Archivo ordinario o regular
d	Directorio
l	Link simbólico (acceso directo)
b	Dispositivo de bloques
c	Dispositivo de caracteres
p	Tubería (pipe)
s	Socket

Los siguientes 9 restantes, representan los permisos del archivo y deben verse en grupos de 3.

Los tres primeros (s₈s₇s₆) representan los permisos para el usuario propietario del archivo. Los tres siguientes (s₅s₄s₃) son los permisos para el grupo propietario del archivo y los tres últimos (s₂s₁s₀) son los permisos para el resto del mundo u otros.

rwx	rwx	rwx
usuario	grupo	otros

El significado de cada opción es:

OPCIÓN	DESCRIPCIÓN
r	read (lectura)
w	write (escritura) En archivos, permiso para modificar En directorios, permiso para crear archivos en el directorio
x	execution (ejecución) En directorios, acceder y listar el contenido

Las 9 posiciones de permisos son en realidad un bit que o está activado (mostrado con su letra correspondiente) o está apagado (mostrado con un guion -).

Ejemplo: los permisos `rw-rw-r--` indican que el propietario (`rw`) puede leer, escribir y ejecutar el archivo, el grupo (o sea los usuarios que estén en el mismo grupo del archivo) (`rw-`) podrá leer y escribir, pero no ejecutar el archivo, y cualquier otro usuario del sistema (`r--`), solo podrá leer el archivo, ya que los otros dos bits de lectura y ejecución no están activados.

Permisos en modo OCTAL

Ahora bien, los permisos también pueden expresarse formato numérico: en octal. Para cada agrupación de 3, el bit:

OPCIÓN	VALOR EN OCTAL
r	$2^2 = 4$
w	$2^1 = 2$
x	$2^0 = 1$

La combinación de bits encendidos o apagados en cada grupo da ocho posibles combinaciones de valores, es decir, la suma de los bits encendidos:

PERMISOS	VALOR	DESCRIPCIÓN
- - -	= 0 (000)	No se tiene ningún permiso
- - x	= 1 (001)	Solo permiso de ejecución
- w -	= 2 (010)	Solo permiso de escritura
- w x	= 3 (011)	Permisos de escritura y ejecución
r - -	= 4 (100)	Solo permiso de lectura
r - x	= 5 (101)	Permisos de lectura y ejecución
r w -	= 6 (110)	Permisos de lectura y escritura
r w x	= 7 (111)	Todos los permisos establecidos, lectura, escritura y ejecución

Cuando se combinan los permisos del usuario, grupo y otros, se obtienen un número de tres cifras que conforman los permisos del archivo o del directorio. Esto es más fácil visualizarlo con algunos ejemplos:

PERMISOS	VALOR	DESCRIPCIÓN
rw- - - -	600	El propietario tiene permisos de lectura y escritura.
rw x -- x -- x	711	El propietario lectura, escritura y ejecución, el grupo y otros solo ejecución.
rw x r-x r-x	755	El propietario lectura, escritura y ejecución, el grupo y otros pueden leer y ejecutar el archivo.

rwX rwX rwX	777	El archivo puede ser leído, escrito y ejecutado por quien sea.
r-- --- ---	400	Solo el propietario puede leer el archivo, pero ni el mismo puede modificarlo o ejecutarlo y por supuesto ni el grupo ni otros pueden hacer nada con él.
rw- r-- ---	640	El usuario propietario puede leer y escribir, el grupo puede leer el archivo y otros no pueden hacer nada.

chmod [opciones] permisos archivo[s]

```
$ chmod 755 informe
$ chmod 511 backup.sh
$ chmod 700 octubre*
$ chmod 644 *
```

Una opción común cuando se desea cambiar todo un árbol de directorios, es decir, varios directorios anidados y sus archivos correspondientes, es usar la opción -R, de recursividad:

```
$ chmod -R 755 datos/*
```

Cambiará los permisos a 755 (rwxr-xr-x) del directorio datos y de todos los subdirectorios y archivos que estén contenidos dentro de este.

Permisos en modo SIMBÓLICO

Otra manera popular de establecer los permisos de un archivo o directorio es a través de identificadores del bit (r, w o x) de los, pero ahora identificando, además:

OPCIÓN	GRUPO	DESCRIPCIÓN
u	TIPOS USUARIOS AFECTADOS (se pueden combinar)	Propietario (user)
g		Grupo de usuarios del propietario (group)
o		Otros usuarios (other)
a		Todos los usuarios (all): usuario, grupo, todos. Equivalente a ugo Equivalente si está en blanco
+	OPERACIONES	Para ampliar permisos
=		Para asignar nuevos permisos (elimina los anteriores)
-		Para eliminar permisos
r	PERMISOS	Lectura
w		Escritura

x	Ejecución
----------	-----------

chmod augo[+|-]rwx[,...] archivo[s]

Ejemplo: si queremos que otros tengan permiso de escritura chmod o+w archivo. Si todos los usuarios deben tener permisos de ejecución chmod a+x archivo.

En este modo de establecer permisos, solo hay que tomar en cuenta que partiendo de los permisos ya establecidos se agregan o se quitan a los ya existentes.

Ejemplos:

ACTUAL	CHMOD	RESULTADO	COMENTARIO
rw- --- ---	a+x	rw- --x --x	Agregar a todos (all) permisos de ejecución.
rw- --x --x	go-x	rw- --- ---	Se eliminan permiso de ejecución para grupo y otros.
rw- r-x r-x	u-x,go-r	rw- --x --x	Al usuario se le quita ejecución, al grupo y otros se le quita lectura.
rw- rw- rw-	u-x,go-rwx	rw- --- ---	Al usuario se le elimina ejecución, al grupo y otros se eliminan todos los permisos.
r-- --- ---	a+r,u+w	rw- r-- r--	A todos se les agrega lectura, al usuario se le agrega escritura.
rw- r-- ---	u-rw,g+w,o+x	--- rw- --x	Al usuario se le eliminan lectura y escritura, al grupo se le agrega lectura y otros se le agrega ejecución.
rw- r-x r-x	u=rw,go=r	rw- r-- r--	Al usuario asigna lectura y escritura, al grupo y otros asigna lectura.

```
$ chmod u+x fichero
$ chmod go-x fichero
$ chmod +x fichero
```

PERMISOS por defecto

Cuando se crea un fichero o un directorio, de forma automática se le asignan unos permisos por defecto. Estos permisos por defecto pueden ser visualizados/modificados a través del comando umask.

umask [mascara]

El comando umask sin parámetros muestra la máscara actual, con parámetros cambia la máscara.

La máscara que se muestra al ejecutar `umask` sin parámetros es la diferencia entre 777 y los permisos que realmente queremos que tenga el fichero/directorio por defecto. De esta forma, si queremos que al crear directorios éstos tengan los permisos 755, entonces la máscara será $777-755=022$.

MODIFICAR el propietario y el grupo

Es posible cambiar al usuario propietario del archivo y al grupo al que pertenece, a través de los comandos:

- **chown** (change owner): cambiar propietario.
- **chgrp** (change group): cambiar grupo.

chown usuario archivo[s]

chgrp grupo archivo[s]

Además, al igual que con `chmod`, también es posible utilizar la opción `-R` para recursividad.

```
$ ls -l presentacion
-rwxrwxr-- sergio ventas      2021-02-01 13:14 presentacion
$ chown juan presentacion
$ ls -l presentacion
-rwxrwxr-- juan  ventas      2021-02-01 13:15 presentacion
$ chgrp gerentes presentacion
$ ls -l presentacion
-rwxrwxr-- juan  gerentes    2021-01-03 15:44 presentacion
```

Permisos ESPECIALES

Los permisos de los archivos se corresponden con un número en octal que varía entre 000 y 777. Sin embargo, existen unos permisos especiales que hacen variar ese número entre 0000 y 7777. Se trata de sticky bit (1000), SGID (2000) y SUID (4000).

- **setuid (4)**: es assignable a ficheros ejecutables y permite que cuando un usuario ejecute dicho fichero, el proceso adquiera los permisos del propietario del fichero ejecutado (usuario efectivo tiene los privilegios del usuario propietario). Se añade 4000 a la representación octal de los permisos del archivo. En la primera terna de permisos se cambia x por s.

El típico uso del bit `setuid` es con `su`, que sirve para ejecutar un fichero con identificadores de grupo y de usuario distintos al del usuario logado, por ello ha de tener este bit activado y así permitir al usuario adquirir temporalmente permisos administrativos para poder hacer el cambio de usuario.

Ejemplo:

```
$ ls -l /bin/su
```

```
-rwsr-xr-x    root root 2021-01-02 15:47 /bin/su
```

Para asignar este bit a un fichero existen dos formas:

```
$ chmod u+s /bin/su
$ chmod 4777 /tmp/file5
-rwsrwxrwx    root root 2021-01-02 15:48 /tmp/file5
```

Y para quitarlo:

```
$ chmod u-s /bin/su
```

- **setgid (2)**: igual que setuid pero escala los privilegios del grupo asignado al fichero/directorio, es decir, el grupo efectivo tiene los privilegios del grupo propietario. Se añade 2000 a la representación octal de los permisos del archivo. En la segunda terna de permisos se cambia x por s.

Ejemplo:

```
$ chmod 2770 /compartido
drwxrws---    root grupo 2021-01-02 16:01 compartido
```

- **sticky bit (1)**: también denominado bit de permanencia este bit suele asignarse en directorios a los que todos los usuarios tienen acceso, y evita que un usuario pueda borrar ficheros/directorios de otro usuario dentro de ese directorio, ya que todos tienen permiso de escritura. La aplicación práctica por excelencia es asignar este bit en /tmp y /var/tmp. Se añade 1000 a la representación octal de los permisos del archivo. En la tercera terna de permisos se cambia x por t.

Ejemplo: tmp tiene permisos 777, el sticky bit se puede asignar del siguiente modo:

```
$ chmod 1777 /tmp
```

O haciendo uso del modo simbólico:

```
$ chmod o+t /tmp
```

Si hacemos un ls veremos la "t" asignada:

```
drwxrwxrwt 13 root root 4096 2021-01-02 16:33 tmp
```

Para eliminar este bit:

```
$ chmod o-t /tmp
```

13. SISTEMAS OPERATIVOS PARA DISPOSITIVOS MÓVILES

Un **SO móvil** es un conjunto de programas de bajo nivel que permite la abstracción de las peculiaridades del hardware específico del teléfono móvil y provee servicios a las aplicaciones móviles, que se ejecutan sobre él.

Destacamos los siguientes sistemas operativos móviles:

- Android.
- iOS.
- Windows 10 Mobile.
- Firefox OS.
- KaiOS.
- Ubuntu Touch.
- Meego.
- Tizen.
- HarmonyOS.
- Sailfish OS.
- Xiaomi HyperOS.

1. Android

Android es un sistema operativo móvil desarrollado por Google. Es uno de los más conocidos junto con iOS de Apple. Está basado en Linux, que junto con aplicaciones middleware está enfocado para ser utilizado en dispositivos móviles como teléfonos inteligentes, tablets, Google TV y otros dispositivos.

Características genéricas:

- Código abierto.
- Núcleo basado en el Kernel de Linux.
- Adaptable a muchas pantallas y resoluciones.
- Utiliza SQLite para el almacenamiento de datos.
- Ofrece diferentes formas de mensajería.
- Navegador web basado en WebKit incluido.
- Soporte de Java y muchos formatos multimedia.
- Soporte de HTML, HTML5, Adobe Flash Player, etc.

- Incluye un emulador de dispositivos, herramientas para depuración de memoria y análisis del rendimiento del software.
- Catálogo de aplicaciones gratuitas o pagas en el que pueden ser descargadas e instaladas (Google Play).
- Bluetooth.
- Google Talk desde su versión HoneyComb, para realizar videollamadas.
- Multitarea real de aplicaciones.

Características incorporadas en **Android 8**:

- Rediseño las **notificaciones** para proporcionar una manera más sencilla y uniforme de administrar su comportamiento y sus configuraciones. Incluye:
 - o Canales de notificación.
 - o Insignias de notificación: exhibición de insignias de notificación en los iconos de inicio de la app. Las insignias de notificación reflejan la presencia de notificaciones asociadas con una app, que el usuario aún no ha descartado ni utilizado. Las insignias de notificación también se conocen como “**puntos de notificación**”.
 - o Posposición: los usuarios pueden posponer las notificaciones para que reaparezcan posteriormente.
 - o Tiempos de espera de notificaciones.
 - o Configuración de notificaciones: para establecer la aparición de texto cuando creas un vínculo a la configuración de notificaciones de tu app desde una notificación.
 - o Descarte de notificaciones: el sistema distingue si un usuario descarta una notificación o si una app la quita.
- **Autofill Framework**: más fácil completar formularios, como los de acceso o tarjetas de crédito. Las apps existentes y las nuevas funcionan con Autofill una vez que el usuario incluye el autocompletado.
- **Modo de imagen en imagen (PIP)**: permite el lanzamiento de actividades en el modo de imagen en imagen. PIP es una clase especial de modo de ventanas múltiples que se usa principalmente para la reproducción de videos.
- **Ajuste automático de tamaño de TextView**: permite configurar el valor de expansión o contracción del texto automáticamente según el tamaño de TextView. Esto significa que es mucho más sencillo optimizar el tamaño de texto en diferentes pantallas o con contenido dinámico.
- **Iconos adaptables**: presenta iconos lanzadores adaptables. Los iconos adaptables admiten efectos visuales y pueden mostrar diferentes formas en los diferentes modelos de dispositivos.
- **Administración de color**: los desarrolladores de apps de imagen para Android ahora pueden aprovechar los nuevos dispositivos que ofrecen pantallas capaces de reproducir una amplia gama de colores. Para mostrar imágenes coloridas, las apps deberán habilitar un indicador en su manifiesto (por actividad) y cargar mapas de bits con un perfil de gran cantidad de colores integrado (AdobeRGB, Pro Photo RGB, DCI-P3, etc.).
- **WebView API**: ofrece varias API para administrar los objetos WebView que muestran contenido web en las apps.
- **Compatibilidad con múltiples pantallas**.

- **Captura de puntero:** para algunas apps, como las de juegos, escritorio remoto y clientes de virtualización, controlar el puntero del ratón representa un gran beneficio.
- **Categorías de apps:** permite que cada app declare una categoría a la que pertenezca, cuando corresponda. Estas categorías se usan para agrupar apps de propósitos o funciones similares cuando se presentan a los usuarios.

Características incorporadas en [Android 9](#):

- **Posicionamiento en interiores con RTT de Wi-Fi:** plataforma compatible con el protocolo de Wi-Fi IEEE 802.11mc, también conocido como tiempo de ida y vuelta de Wi-Fi (RTT). Las apps podrán usar RTT API para medir la distancia a los puntos de acceso (PA) WiFi cercanos con capacidad para RTT.
- **Compatibilidad con recorte de pantalla.**
- **Notificaciones:** mejoras en la experiencia de mensajería: compatibilidad simplificada con participantes de conversaciones y compatibilidad con imágenes.
- **Configuraciones de canales, transmisiones y el modo No interrumpir:** permiten crear un canal que el usuario puede personalizar para cada tipo de notificación que desees mostrar.
- **Compatibilidad con varias cámaras y actualizaciones de cámaras.**
- **ImageDecoder para elementos de diseño y mapas de bits:** clase que proporciona un enfoque modernizado para la decodificación de imágenes.
- **Animación:** se introduce la clase AnimatedImageDrawable para dibujar y mostrar imágenes animadas GIF y WebP.
- **Video HDR VP9, compresión de imagen HEIF y API de medios:** incorpora compatibilidad integrada con rango dinámico alto (HDR) VP9 Perfil 2 para ofrecer películas en HDR a través de YouTube, Play Movies u otros.
- **Autofill Framework:** se introducen varias mejoras que los servicios de autocompletar.
- **Diálogo de autenticación biométrica unificada:** para que puedan hacer uso las apps.

Características incorporadas en [Android 10](#):

- **Navegación por gestos:** los usuarios pueden habilitar la navegación por gestos en todo el dispositivo.
- Mejoras en **privacidad y seguridad.**
- **TLS 1.3** habilitado de forma predeterminada.
- **Tema oscuro:** se aplica a todo el sistema y es ideal para los momentos en los que hay poca luz y ayuda a ahorrar batería. Los usuarios pueden activarlo desde Configuración o en Ahorro de batería.
- **Burbujas flotantes:** para las notificaciones.

Características incorporadas en [Android 11](#):

- **Permisos de un solo uso** para las apps.
- **Controles de dispositivos de dispositivos externos conectados.**
- **Menú apagado renovado.**

Características incorporadas en [Android 12](#):

- **Experiencia de usuario:** efecto de desplazamiento lateral ampliado, pantallas de presentación de app, resolución de intents web (solicitar una acción de otro componente), mejora en el modo envolvente para la navegación por gestos, apps en modo multiventana, vista previa de la cámara en pantallas grandes...
- **Material You:** lenguaje de diseño nuevo.
- **Rendimiento:** intervalo restringido de App Standby.
- **Seguridad y privacidad:** botones de activación del micrófono y de la cámara, indicadores de acceso al micrófono y a la cámara, notificaciones de acceso al portapapeles, apps no pueden cerrar los diálogos del sistema, bloque de eventos táctiles que no son de confianza.
- **Imágenes y gráficos:** mejora del cambio de frecuencia de actualización.
- **Conectividad:** actualizaciones de Passpoint.

Características incorporadas en [Android 13](#):

- **Habilitar iconos de apps temáticos:** con esta función, se ajusta el tono de los iconos de apps en los launchers de Android compatibles para heredar el color del fondo de pantalla y otros temas que eligió el usuario.
- **Preferencias de idioma de las apps.**
- Compatibilidad con renderización para las fuentes de COLR.
- API de configuración rápida del panel de notificaciones que permite a los usuarios cambiar la configuración o realizar acciones rápidas sin salir del contexto de una app.
- **Vista previa del portapapeles.**
- **Gesto de retroceso predictivo.**
- Compatibilidad con el estándar MIDI 2.0.
- Preferencia de accesibilidad para todo el sistema que permite a los usuarios habilitar audiodescripciones en todas las apps.
- Actualización de las bibliotecas principales de Android para alinearlas con la versión **OpenJDK 11**.
- Las APIs de Camera2 admiten la captura de video de alto rango dinámico (HDR).
- Compatibilidad con el esquema de firma de **APK v3.1**.

Características incorporadas en [Android 14](#):

- **API de Grammatical Inflection**, que permite agregar compatibilidad con los usuarios que hablan idiomas con géneros gramaticales.
- Paquetes de **metadatos de app** que proporcionan los desarrolladores de una forma transparente para incluir información sobre ellos, la app y la forma en que recopilan, comparten y protegen los datos del usuario.
- Conectividad por **satélite**.
- Estado físico de forma **segura** con Health Connect.
- Escalamiento de fuente no lineal al 200%.
- Detección de capturas de pantalla de los usuarios.

- Animaciones predictivas dentro de la app.
- Compartir pantalla de una app.
- Soporte de Ultra HDR para imágenes.
- Actualizaciones de OpenJDK 17.

Características incorporadas en [Android 15](#):

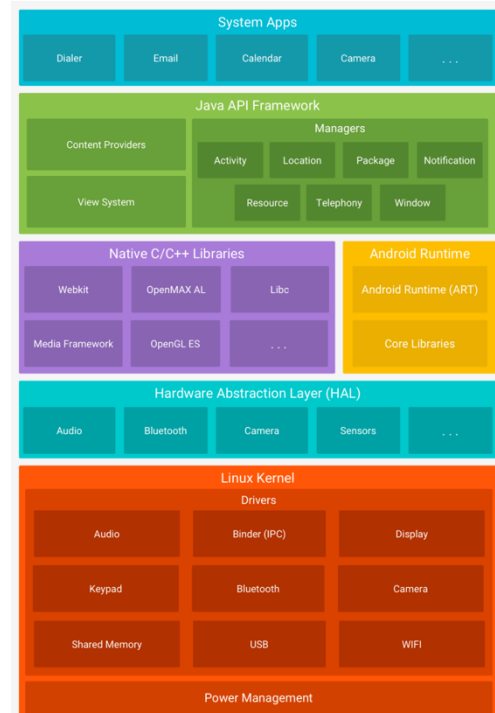
- El **espacio privado** permite a los usuarios crear un espacio separado en sus dispositivos, en el que pueden mantener las apps sensibles, con una capa adicional de autenticación.
- **Uso compartido de pantalla parcial** para que los usuarios puedan compartir o grabar solo la ventana de una app en lugar de toda la pantalla del dispositivo.
- **Archivado apps** guardando la información de configuración y archivos para un posterior desarchivado.
- **Actividades en segundo plano seguras** para evitar aumento sus privilegios y abuso de la interacción del usuario.
- Mejora de la **compatibilidad en la conectividad por satélite** admitiendo SMS, MMS y RCS.
- Mejora con **NFC mediante el modo observación**, en el que se escucha pero no se responde a los lectores de NFC.
- **Vibraciones enriquecidas para las notificaciones entrantes**, lo que permite al usuario distinguir entre distintos tipos de notificaciones sin tener que mirar su dispositivo.
- Identificación de canciones.

Características incorporadas en [Android 16](#):

- Rediseño de la interfaz con Material 3 Expressive.
- **Medición de distancia** con seguridad mejorada mediante la compatibilidad con funciones de seguridad sólidas en la ubicación Wi-Fi en dispositivos compatibles con Wi-Fi 6 802.11az, lo que permite que las apps combinen la mayor precisión, la mayor escalabilidad y la programación dinámica del protocolo con mejoras de seguridad, como la encriptación basada en AES-256 y la protección contra ataques de intermediarios.
- **APIs de rango genéricas**, a través de RangingManager, que proporciona formas de determinar la distancia y el ángulo en el hardware compatible entre el dispositivo local y un dispositivo remoto.
- Soporte para grabación de video con el códec (**APV**), que está diseñado para usarse en la grabación de video de alta calidad a nivel profesional.
- Optimizaciones de la **cuota de JobScheduler** ajustando la cuota de tiempo de ejecución de trabajos en función de algunos factores.
- **Medición de texto vertical**.
- **Privacy Sandbox** para garantizar a los usuarios su privacidad.
- API de **uso compartido de claves de Android Keystore** con otras apps.
- **Audífonos LEA**: se ofrecen controles de volumen ambiental y la posibilidad de usar el teléfono como micrófono.

Arquitectura:

- **Aplicaciones:** conjunto de apps centrales, entre las que se encuentran, un cliente de correo electrónico, programa de SMS, calendario, mapas, navegador, contactos y otros. Todas ellas escritas en Java.
- **Marco de trabajo de aplicaciones:** los desarrolladores tienen acceso completo a los mismos APIs del framework (escritos en Java) usados por las aplicaciones base. La arquitectura está diseñada para simplificar la reutilización de componentes; cualquier aplicación puede publicar sus capacidades y cualquier otra aplicación puede luego hacer uso de esas capacidades.
- **Bibliotecas:** incluye un conjunto de bibliotecas de C/C++ nativas usadas por varios componentes del sistema.
- **Runtime de Android:** incluye un conjunto de bibliotecas base que proporcionan la mayor parte de las funciones disponibles en las bibliotecas base del lenguaje Java. Cada aplicación Android corre su propio proceso, con su propia instancia de la **máquina virtual ART** (Android Runtime). Antes la versión 5.0 Lollipop, la máquina virtual era **Dalvik**.
- **Capa de abstracción de hardware (HAL):** ofrece interfaces estándares que exponen las capacidades de hardware del dispositivo a la API de Java de nivel más alto. La HAL consiste en varios módulos de biblioteca y cada uno de estos implementa una interfaz para un tipo específico de componente de hardware (como el módulo de la cámara o de Bluetooth). Cuando la API realiza una llamada para acceder al hardware del dispositivo, el sistema Android carga el módulo de biblioteca para el componente de hardware en cuestión.
- **Núcleo Linux:** la base de la plataforma Android es el kernel de Linux y depende de él para los servicios base del sistema como seguridad, gestión de memoria, gestión de procesos, pila de red y modelo de controladores. También actúa como capa de abstracción entre el hardware y el resto de la pila de software.



Versiones de Android:

Letra	Nombre en clave	Número versión	Fecha lanzamiento
A	Apple Pie	1.0	23/09/2008
B	Banana Bread	1.1	09/02/2009
C	Cupcake	1.5	25/04/2009
D	Donut	1.6	15/09/2009
E	Eclair	2.0 – 2.1	26/10/2009
F	Froyo	2.2 – 2.2.3	20/05/2010
G	Gingerbread	2.3 – 2.3.7	06/12/2010
H	Honeycom	3.0 – 3.2.6	22/02/2011
I	Ice Cream Sandwich	4.0 – 4.0.5	18/10/2011
J	Jelly Bean	4.1 – 4.3.1	09/07/2012
K	KitKat	4.4 – 4.4.4	31/10/2013
L	Lollipop	5.0 – 5.1.1	12/11/2014
M	Marshmallow	6.0 – 6.0.1	05/10/2015
N	Nougat	7.0 – 7.1.2	15/06/2016
O	Oreo	8.0 – 8.1	21/08/2017
P	Pie	9.0	06/08/2018
	Android 10 (Quince Tart)	10.0	03/09/2019
	Android 11 (Red Velvet Cake)	11.0	08/09/2020
	Android 12 (Snow Cone)	12.0	04/10/2021
	Android 13 (Tiramisú)	13.0	15/08/2022
	Android 14 (Upside Down Cake)	14.0	04/10/2023
	Android 15 (Vanilla Ice Cream)	15.0	15/10/2024
	Android 16 (Baklava)	16.0	10/06/2025
	Android 17 (Cinnamon Bun)	17.0	En desarrollo

La distribución de actualizaciones **OTA** (Over-The-Air) es una evolución para servir actualizaciones de software o firmware a un dispositivo móvil mediante una conexión inalámbrica.

Edición Go

Android edición Go es un sistema operativo para dispositivos móviles diseñado para smartphones **básicos** con menos RAM que los dispositivos de alta gama. Android edición Go tiene un menor coste de funcionamiento y ahorra datos, lo que permite que los fabricantes de equipos originales compilen dispositivos básicos y asequibles que amplíen las posibilidades de las personas.

Existen requisitos mínimos de RAM para que los dispositivos ejecuten Android edición Go, según la versión concreta:

Lanzamiento	Android 8.1	Android 9	Android 10	Android 11	Android 12	Android 13
RAM mínima	512 MB	512 MB	512 MB	1 GB	1 GB	2 GB

Capas de personalización

Permiten a las empresas añadir funciones adicionales y específicas a las ofrecidas por Android para adaptarse a los dispositivos hardware. Destacan las siguientes:

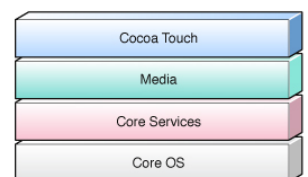
- **One UI** de Samsung.
- **EMUI** de Huawei.
- **MIUI** de Xiaomi.
- **OxygenOS** de OnePlus.
- **ColorOS** de Oppo.
- **OriginOS** de Vivo.

2. iOS

iOS es un sistema operativo móvil de la empresa Apple Inc. Originalmente desarrollado para el iPhone (iPhone OS), después se ha usado en dispositivos como el iPod touch y el iPad. No permite la instalación de iOS en hardware de terceros. La versión actual es **iOS 26**.

iOS deriva de macOS (sistema operativo para PC's y portátiles) que a su vez está basado en Darwin BSD. Está dividido en **4 capas**:

- **Core OS**: núcleo del sistema. Se encarga de la seguridad, memoria, manejo de ficheros...
- **Core Services**: servicios principales del dispositivo: bases de datos, red...
- **Media**: desarrollada principalmente en Objective C con tecnologías para acceso a ficheros multimedia.
- **Cocoa Touch**: capa con componentes visuales con la que interactúa el usuario.



iOS utiliza el formato de archivo .ipa para las aplicaciones. **IPA** son las siglas de iOS AppStore Package.

3. Windows 10 Mobile

Microsoft lanzó Windows Phone como predecesor de Windows 10 Mobile, sistema actual. En la actualidad, Microsoft (10/12/2019) ha dejado de actualizar y mantener Windows 10 Mobile y, por tanto, no presta más soporte. Esto supone, la desaparición del mismo por desuso.

4. Firefox OS

Firefox OS es un sistema operativo móvil basado en HTML5 con núcleo Linux, de código abierto para varias plataformas y desarrollado por Mozilla.

Ha sido creado en una plataforma abierta, utilizando núcleo Linux para arrancar sobre el mismo motor del navegador Firefox, el motor web “Gecko”. Concretamente, Firefox OS consta de 3 capas:

- **Gaia (interfaz de usuario)**: La interfaz de usuario de la plataforma Firefox OS. Cualquier cosa dibujada en la pantalla una vez que Firefox OS ha sido iniciado es un producto de la capa Gaia. La misma implementa la pantalla de seguridad (lock screen), pantalla principal (home screen) y todas las aplicaciones estándares de cualquier teléfono inteligente moderno. Gaia se implementa en su totalidad empleando HTML, CSS y JavaScript. Las interfaces subyacentes al sistema operativo son Web APIs de código abierto, que se implementan por medio de la capa Gecko. Las aplicaciones de terceros se pueden instalar en paralelo con la capa Gaia.
- **Gecko (motor de renderizado web)**: Este es el runtime de aplicaciones de Firefox OS, es decir, la capa que provee todo el soporte para el trío de estándares de código abierto: HTML, CSS y JavaScript. Es responsable de que esas APIs funcionen bien en cualquier sistema operativo soportado por Gecko. Esto implica que Gecko incluya, además de otras cosas, paquetes de redes y de gráficos, un motor de diagramación, una máquina virtual de JavaScript y capas de adaptación (porting).
- **Gonk (núcleo)**: Gonk es el sistema operativo de bajo nivel de la plataforma Firefox OS que consiste un núcleo/kernel Linux (basado sobre el Android Open Source Project (AOSP)) y una capa de abstracción de hardware de espacio de usuario (HAL por su sigla en inglés). Gonk es una distribución de Linux muy sencilla.

5. KaiOS

Sistema operativo móvil basado en Linux, sucesor de Firefox OS (descontinuado) y derivado de B2G (Boot to Gecko). Destinado a móviles básicos no táctiles y de bajo coste.

6. Ubuntu Touch

Ubuntu Touch es un sistema operativo móvil basado en Linux desarrollado por Canonical. Más tarde, UBports lo adoptó y comenzó a desarrollarlo como un proyecto comunitario.

7. MeeGo

Predecesor de Tizen, es un sistema operativo para móviles impulsado por Linux Foundation.

8. Tizen

Tizen es un sistema operativo abierto y flexible creado desde cero para abordar las necesidades de todas las partes interesadas del ecosistema de dispositivos móviles y conectados, incluidos los fabricantes de dispositivos, operadores móviles, desarrolladores de aplicaciones y proveedores de software independientes. Tizen está desarrollado por una comunidad de desarrolladores, bajo un gobierno de código abierto, basado en Linux y está abierto a todos los miembros que deseen participar.

9. HarmonyOS

Sistema operativo de código abierto desarrollado por Huawei que es multiplataforma, lo que permite que corra en dispositivos móviles pero también en cualquier otro dispositivo IoT (Internet of Things) como smartwatches, auriculares o tablets. Versión actual 5.

Operativo inicialmente solo en China para dispositivos móviles, en 2023 disponible a nivel mundial.

10. Sailfish OS

Sistema operativo independiente (sin vínculos con grandes organizaciones) y de código abierto desarrollado por la empresa finlandesa Jolla tanto para tabletas como para dispositivos móviles.

11. Xiaomi HyperOS

Sistema operativo móvil basado en Android, evolución de la capa de personalización MIUI.

12. Otros SO móviles

AliOS, Symbian OS y BlackBerry OS.

